CS2109S: Introduction to AI and Machine Learning

Lecture 3:
Local and Adversarial Search

27 January 2026

Capstone Project

Objective: Apply the AI & ML knowledge that you have/will learn throughout the course.

- Develop an agent to play a grid puzzle game.
  - Fully-observable, deterministic, single-agent.
  - Image observation.
- Can use any techniques: search and/or ML.
- Details regarding timeline, grading, rules will be given on release.
- Due date: end of semester (week 13).

Will be released sometime this week (hopefully)!

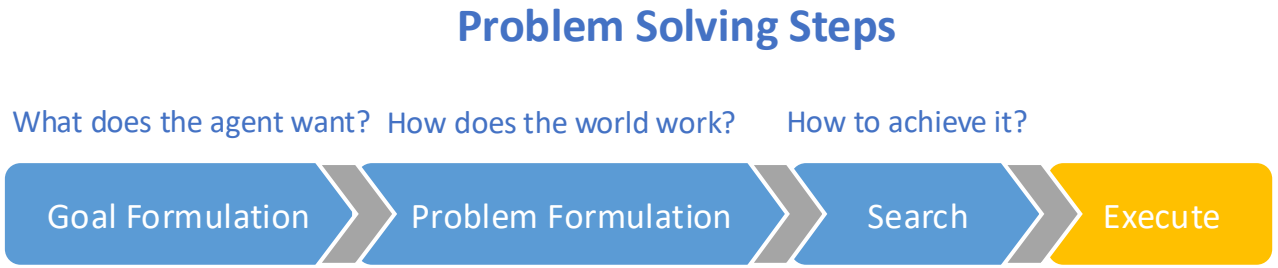
Outline

- **Local Search**
  - Problem Formulation
  - Hill-climbing
- Adversarial Search
  - Problem Formulation
  - Minimax
  - Alpha-beta Pruning
  - “Real-world” Games

Recall: Designing an Agent

To solve a problem using search, the agent needs to have:

- A goal, or a set of goals
- A model of the environment
- A search algorithm

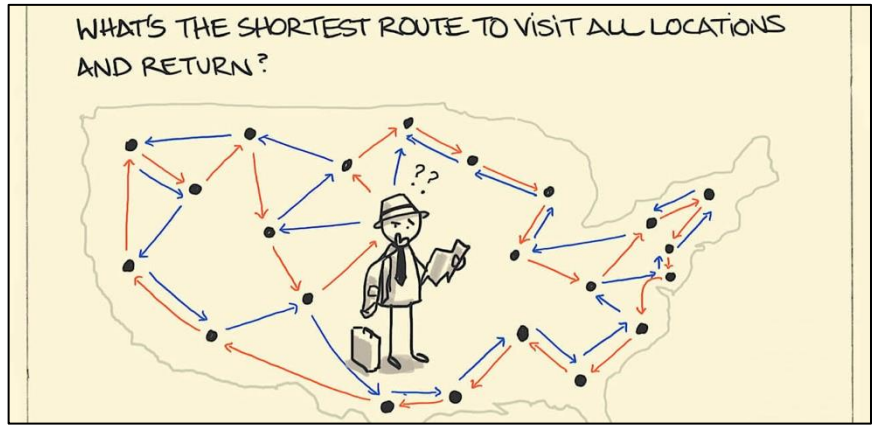


Difficult Search & Optimization Problems

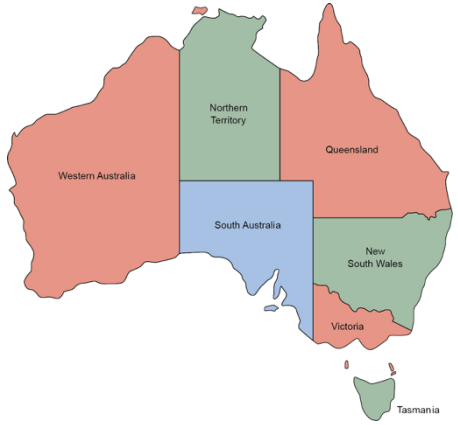
- **Combinatorial optimization problems**
  - Example: Travelling Salesperson Problem (TSP)
- **Constraint Satisfaction Problems (CSPs)**
  - Example: Graph coloring problem, scheduling problems
- **Continuous Optimization Problems**
  - Example: non-linear non-convex programming problems

Many of these problems are NP-Hard!

NP-hard: at least as hard as any problem in NP.



Credit: sketchplanations



Credit: Classic Computer Science Problems in Java, David Kopec

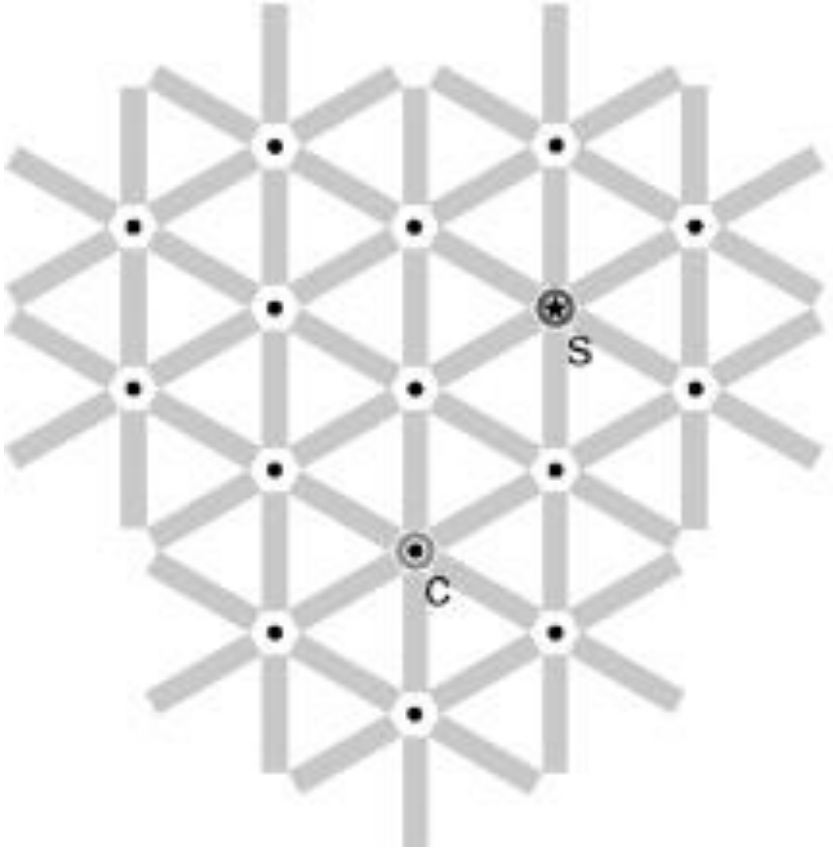
Previously: Systematic Search

- Traverse the state space in a **systematic** manner
- Typically, **complete** and **optimal** under certain assumptions
  - E.g., BFS, UCS, IDS, A*
- Solving difficult search and optimization problems with this paradigm is intractable: not solvable within a reasonable amount of time

Local Search

- Simple Strategy: Traverse the state space by considering only the best **locally-reachable** state:
  - Start at a (random) position in a state space.
  - Consider neighboring states of current state.
    - Move to “better” neighboring state, while “forgetting” the other neighboring states.
    - Iterate!
  - **The solution is the final state after a certain number of steps.**
- Typically, **incomplete** and **suboptimal**
- **Any-time** property: longer runtime, (often) better solution
  - Can obtain a “good enough” solution for difficult search and optimization problems

Local Search



Holger H. Hoos, Thomas Stützle, in Stochastic Local Search, 2005

Recall:

Problem Formulation in Systematic Search

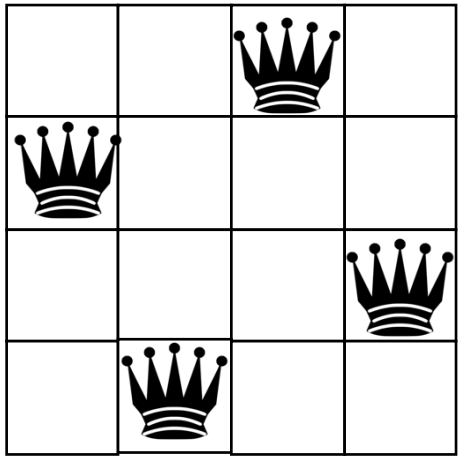
- **States:** Each state corresponds to an actual/specific situation or configuration of the problem domain.

- **Initial State:** The starting configuration
- **Goal Test / State(s)**
- **Actions**
- **Transition model**
- **Action cost function**

Problem Formulation in Perturbative Local Search

- **States:** Represent different configurations or candidate solutions. They may or may not map to an actual problem state but are used to represent potential solutions.
- **Initial State:** The starting configuration (candidate solution).
- **Goal Test (optional):** Check if a current state is the desired solution.
- **Successor Function:** A function that generates neighboring states (candidate solutions) by applying modifications to the current state.

Example: N-Queens



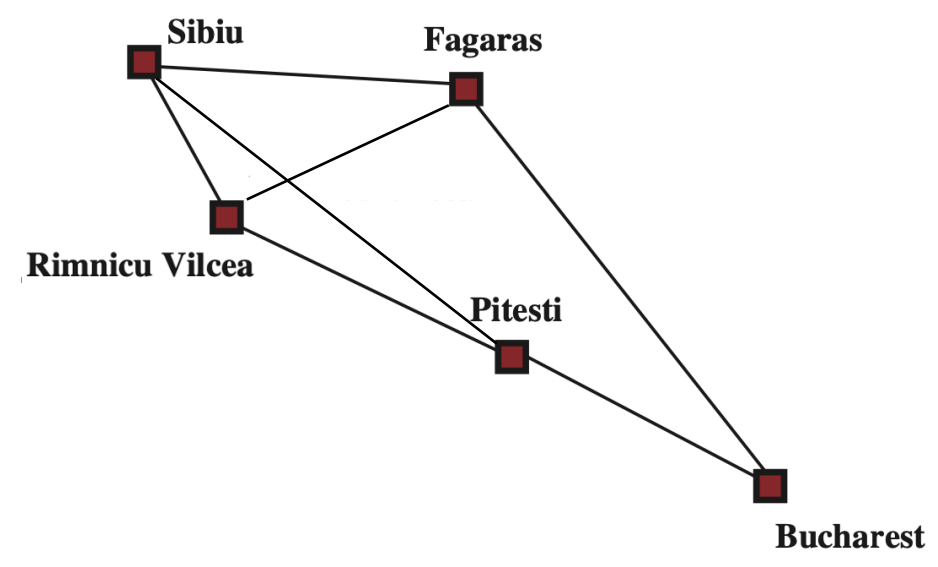
States	Configurations of queens on the board
Initial state	Random placement of queens, one queen on each column
Goal test	No two queens threaten each other, i.e., no two queens share the same row, column, or diagonal
Successor function	Move a queen to a different row within the same column

Example: Minimizing a Function

Find an **integer** x that minimizes a "black box" function $f(x)$

States	All possible integers
Initial state	Random integer
Goal test	None
Successor function	Make change to x, either by increasing or decreasing it by 1

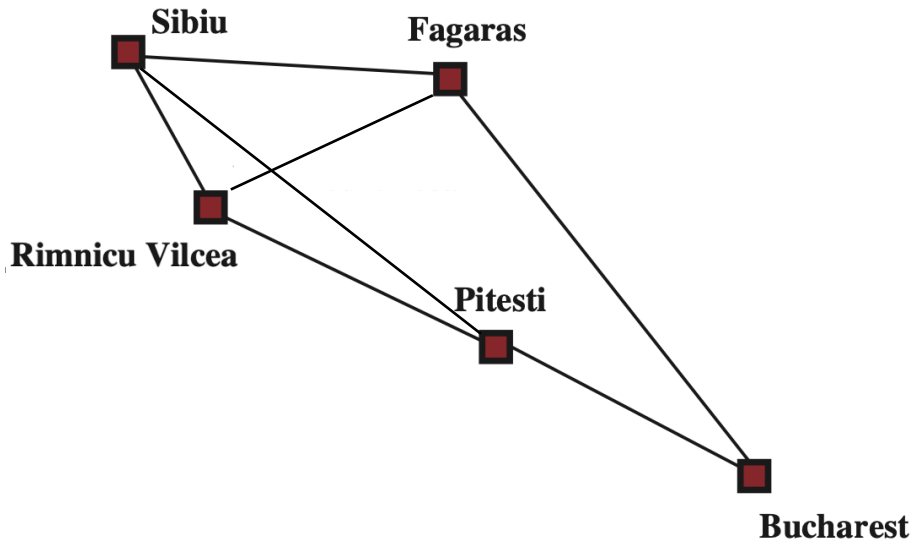
PollEverywhere



In local search, the solution is the final state rather than the path taken to reach it. Can local search be used to potentially find a good solution for shortest path problems?

A. Yes
B. No
C. It depends

PollEverywhere



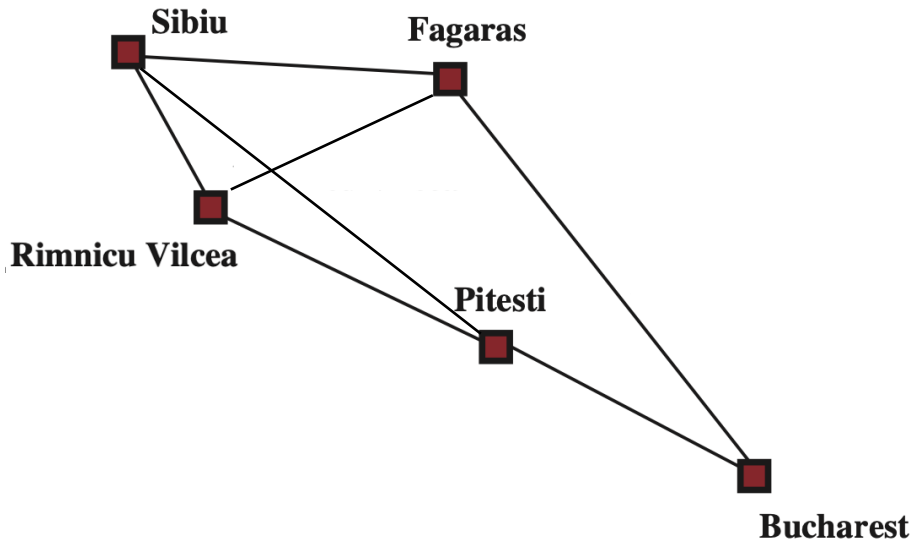
In local search, the solution is the final state rather than the path taken to reach it. Can local search be used to potentially find a good solution for shortest path problems?

A. Yes

B. No

C. It depends

Example: Path Finding



States	Paths from Sibiu to Bucharest
Initial state	Random path from Sibiu to Bucharest
Goal test	None
Successor function	Replace a sub-path between any arbitrary two cities in the current path with other valid paths

Example:

- Sibiu – Fagaras – Rimnicu Vilcea – Pitesti – Bucharest

↓

- Sibiu – Fagaras – Bucharest
- Sibiu – Fagaras – Rimnicu Vilcea – Sibiu – Pitesti – Bucharest

Evaluation Function

An **evaluation function** (also known as a **fitness function** or **objective function**) is a mathematical function used to assess the quality or desirability of a state.

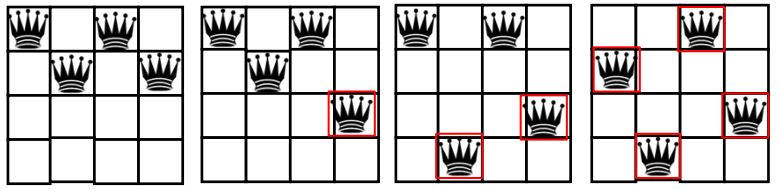
The function takes in a state as an argument and produces a number.

```
value = eval(state)
```

Depending on the problem and the algorithm, we may want to either minimize or maximize this function.

Evaluation Function - Examples

- In the **n-queens problem**, an evaluation function can be the number of "safe" queens, i.e., queens not attacking each other in the same row, column, or diagonal. The goal is to maximize this number.



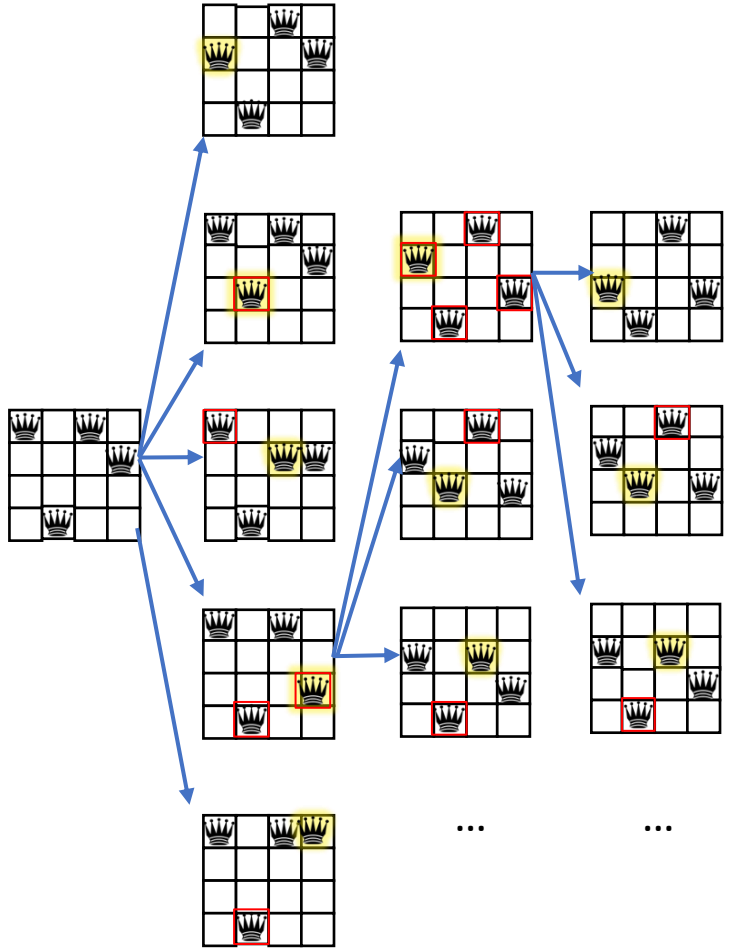
- In the problem of **minimizing a function**, the evaluation function is the (objective) function itself, i.e., $f(x)$. The goal is to minimize it.

Note: A minimization objective $f(x)$ can be transformed into a maximization objective by defining a new objective: $\hat{f}(x) = -f(x)$, and vice versa.

Hill climbing algorithm

(Steepest Ascent Search, Greedy Local Search)

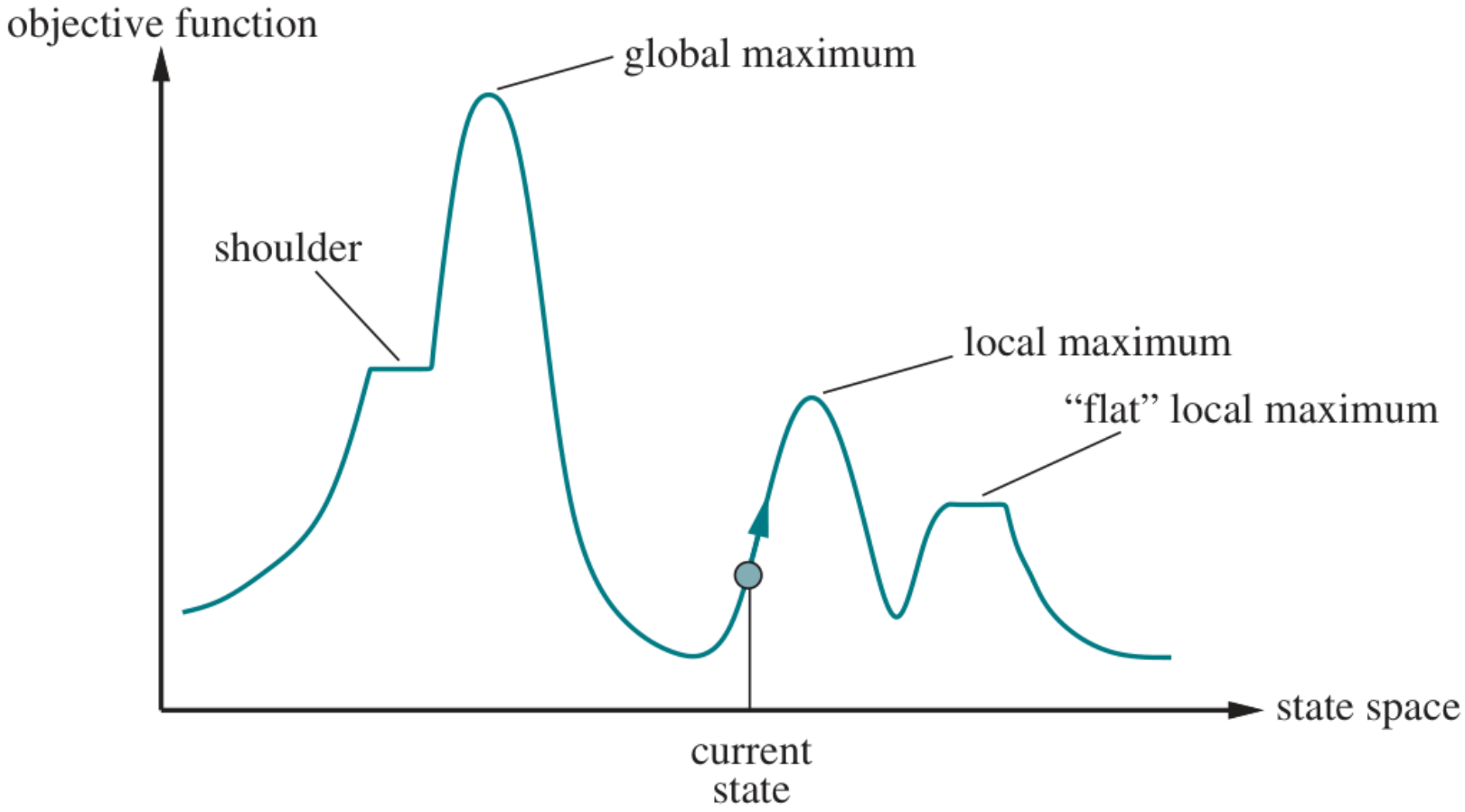
```
current_state = initial_state
while True:
    best_successor = a highest-eval successor state of current_state
    if eval(best_successor) <= eval(current_state):
        return current_state
    current_state = best_successor
```



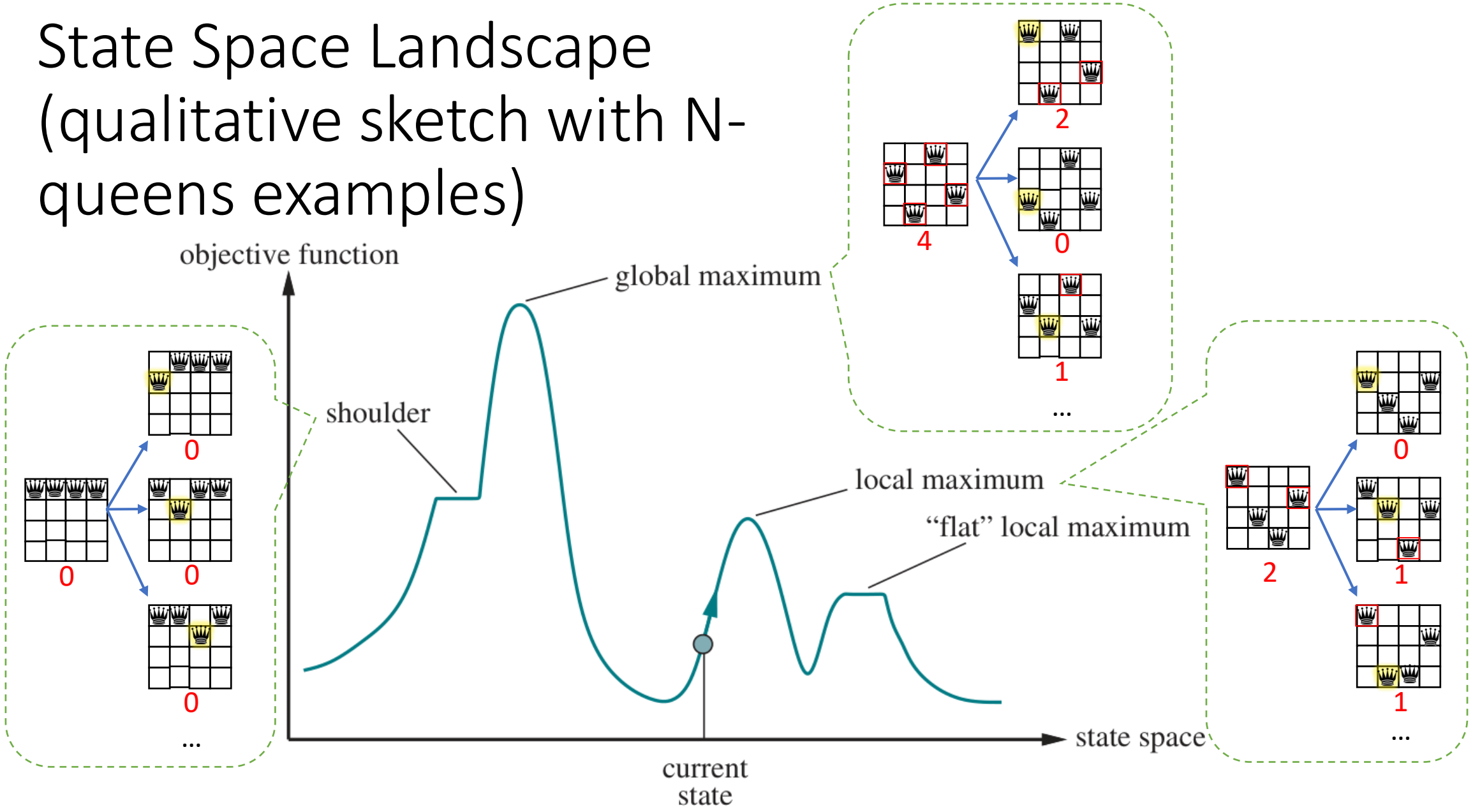
State Space Landscape

- **Global Maximum**: The overall **highest point** or solution across the entire state space that represents the optimal solution.
- **Local Maximum**: A point in the state space that is **higher than its immediate neighbors**, but there may be a higher value elsewhere in the state space. It represents a local optimum, not necessarily the best solution globally.
- **Shoulder**: A region in the state space where the objective function has a **flat** area, making it difficult for the algorithm to move past it towards a better solution.

State Space Landscape
(for example: black-box function maximization)



State Space Landscape (qualitative sketch with N-queens examples)



Break

A BRIEF HISTORY OF CHESS

TEDEd

Outline

- Local Search
  - Problem Formulation
  - Hill-climbing
- **Adversarial Search**
  - Problem Formulation
  - Minimax
  - Alpha-beta Pruning
  - “Real-world” Games

Multi-agent Problems

Multi-agent problems involve **multiple agents** interacting within an environment, where each agent has its own goal(s) and decision-making process. The problems could be:

- **Cooperative:** Agents work together towards a common goal
  - Example: robotic swarms
- **Competitive:** Agents have opposing goals
  - Example: chess, go
- **Mixed-Motive:** Some cooperation and some competition
  - Example: auctions, the Prisoner's Dilemma
- …

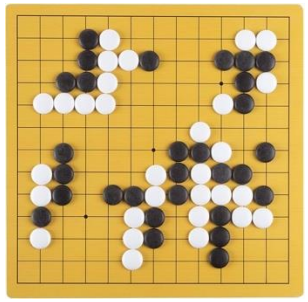
Competitive Multi-agent Problems

Conflicting Goals: Agents have opposing goals, and one agent's success might directly impact the failure of the other.

Example:

- **Adversarial Games**: Players (agents) compete against each other.
  - **Two-player zero-sum games**: one player's gain is the other player's loss. Example: chess, go, tic-tac-toe, 2-player poker.
  - …
- …

Credit: chess.com

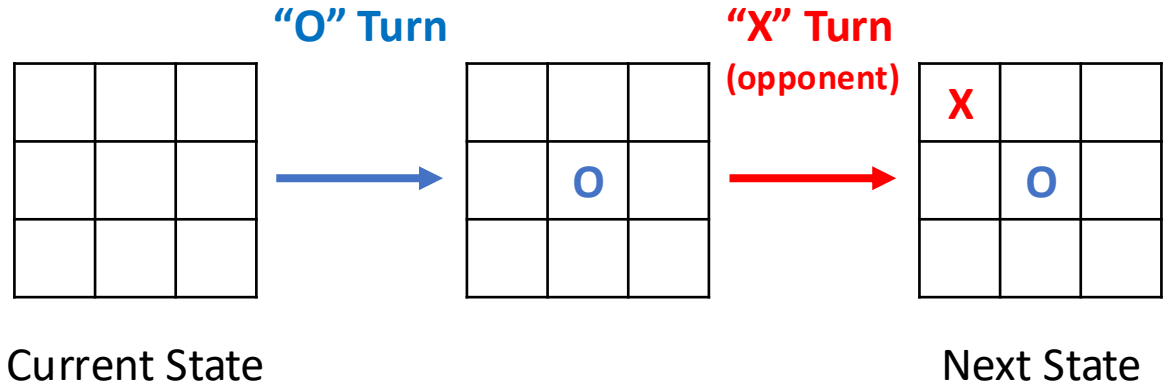


Credit: Amazon.sg

Credit: Wikipedia

Turn-Taking Games

The next state of the environment depends <u>not only</u> on our action in the current turn but also on the action of other agents in the next turn, which are assumed to be strategic, i.e., the other agents try to reach their goal(s).

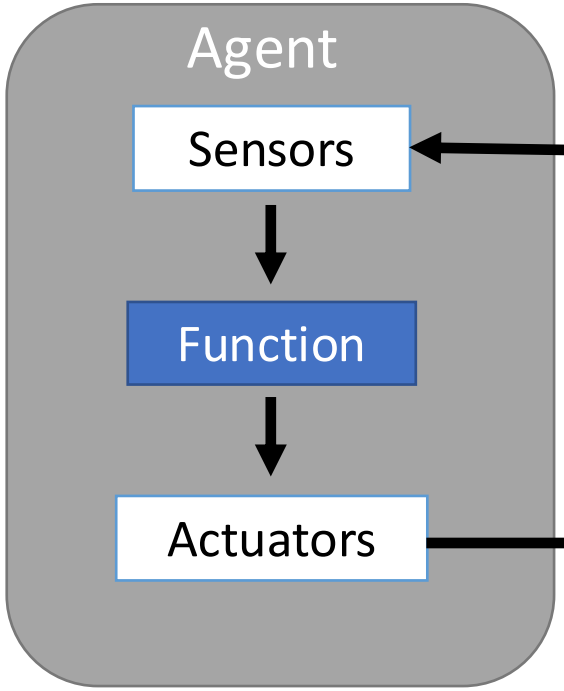


Designing an Agent

for **competitive multi-agent problems** (i.e., "classical" **adversarial games**)

Utility-based agent

Adversarial search



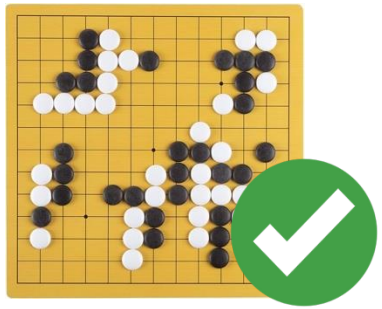
percepts

actions

Credit: chess.com

Fully observable
Deterministic
Discrete

No infinite runs
Two-player zero-sum
Turn-taking



Credit: Amazon.sg

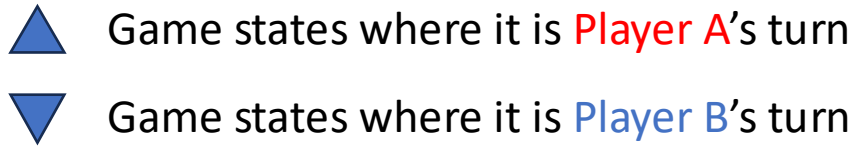
Credit: Wikipedia

Credit: Toys R Us

Games – Terminology

- **Player** (Agent): An entity making decisions or taking actions in a game.
- **Turn**: A phase where a player makes a move, after which control passes to the next player.
- **Move** (Action): A decision or action by a player that changes the game state.
- **End State** (Terminal State): A state where the game ends, with no further moves possible.
- **Winning Condition**: Criteria that define when a player wins the game.

Games – Game Tree



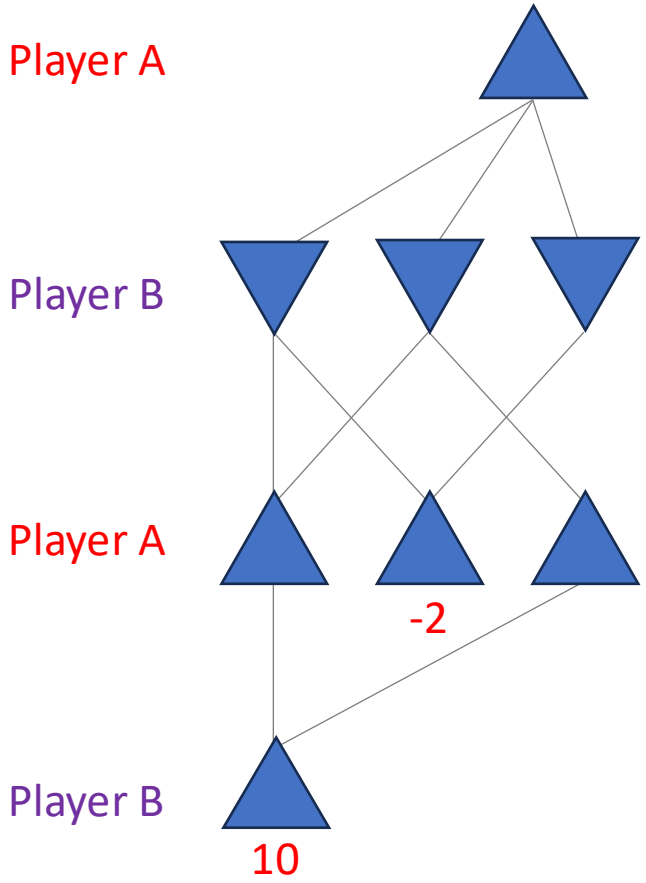
A game tree is a graphical representation of all possible states and moves in a game.

In this diagram:

- **Nodes** indicate possible game states.
- **Edges** show legal moves or transitions between states.

The **outcome of the game** (such as monetary payoff, win, loss, or draw) is displayed beneath each terminal node (end state).

The outcome is from the perspective of Player A.



Recall:

Problem Formulation in Systematic Search

- **States**
- **Initial State**
- **Goal Test / State(s)**

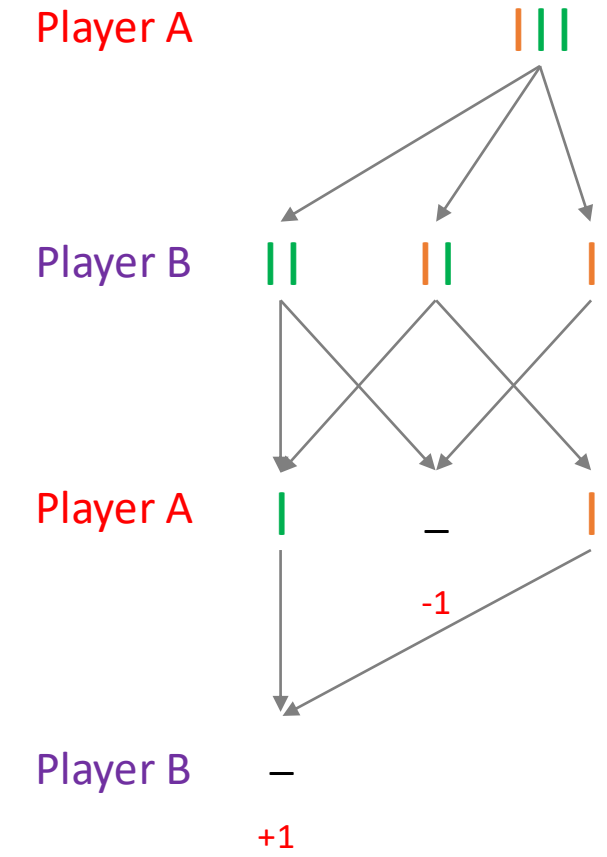
- **Actions**
- **Transition**
- **Action cost function**

Problem Formulation in Adversarial Search

- **States**
- **Initial State**
- **Terminal States** : states where the game ends, such as win/lose/draw states.
  - This set of states could be defined as a function.
- **Actions**
- **Transition**
- **Utility Function**: output the value of a state from the perspective of our agent.
  - Our agent (player A) wants to maximize the utility.
  - In games such as Chess or tic-tac-toe, the outcome for player A is a win, loss, or draw.
    - Assign utility values of +1, -1, and 0, respectively, to represent these outcomes.
  - In other games such as Go with territory scoring, the outcome for player A is some number.
    - Assign integer/real-valued utility values to the outcomes.
  - The utility of non-terminal states is typically not known in advance and must be computed.

Example – Sticks

- Two piles of sticks (green and orange) are given
- Player can take any number of sticks from a single pile
- The player who cannot make any move loses

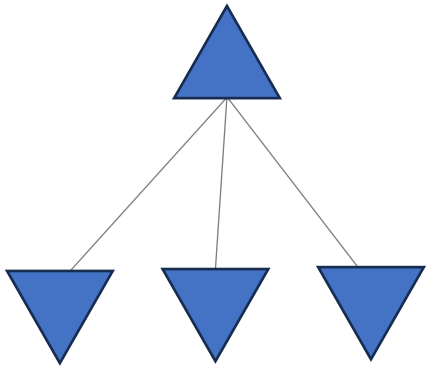


States	Configurations of piles
Initial state	A number of green sticks and orange sticks
Terminal states	No stick
Actions	Take 1,2,… sticks from one of the piles
Transition	Remove the sticks from the pile
Utility function	+1, 0, -1 for win, draw, lose, respectively

Q: Which move would you choose?

Assume you are Player A and the game is in some state. Which move?

Player A



Minimax – Overview

Algorithm for **two-player zero-sum** game

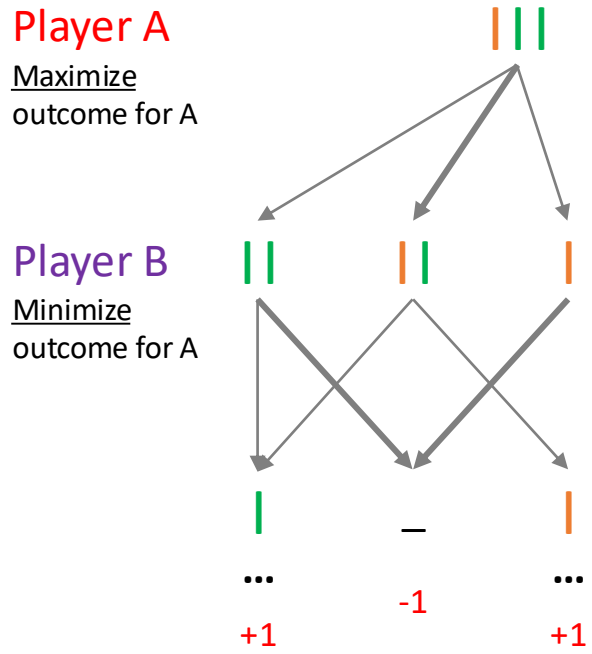
Core assumption: all players play optimally

Take the view of player A:

Try to maximize the outcome of the game

Player B tries to minimize the outcome of Player A, i.e., maximize their own outcome

Minimax algorithm computes the outcome of the game in a depth-first manner.



Minimax – Functions

Expand function **expand**(state)

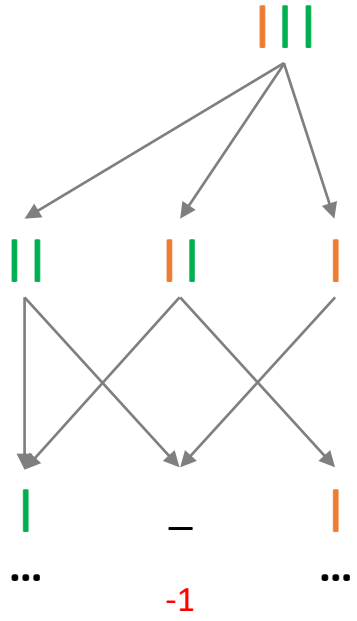
- For each action, compute: next_state = **transition**(state, action)
- Returns a set of (action, next_state) pairs.

Terminal function **is_terminal**(state)

- Returns true/false if the state is a terminal state.

Utility function: **utility**(state)

- Returns a score for the state from the point of view of Player A.

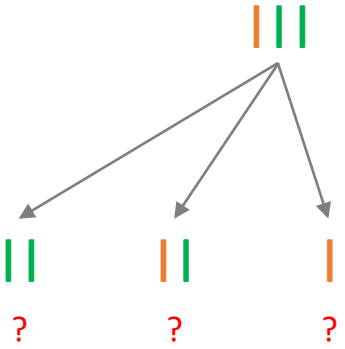


Minimax

```
def minimax(state):
    (action, v) = max_value(state)
    return action
```

Player A

Choose action with max value for A



Note: We assume here that **max_value** also returns the action to which the value v corresponds to. We do not write this explicitly in the following pseudo-code.

Minimax

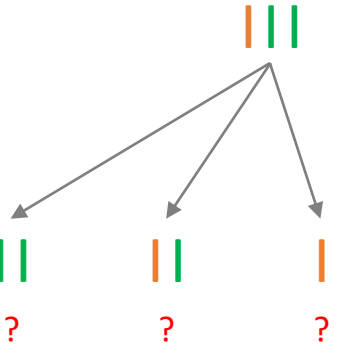
```
def minimax(state):
    (action, v) = max_value(state)
    return action
```

```
def max_value(state):
    if is_terminal(state): return utility(state)
    v = -∞ // smallest value possible
    for next_state in expand(state):
        v = max(v,  minimum value for player A in the next_state)
    return v
```

Assumes opponent play **optimally**:
trying to **minimize** player A value

Player A

Choose action with max value for A



Player B

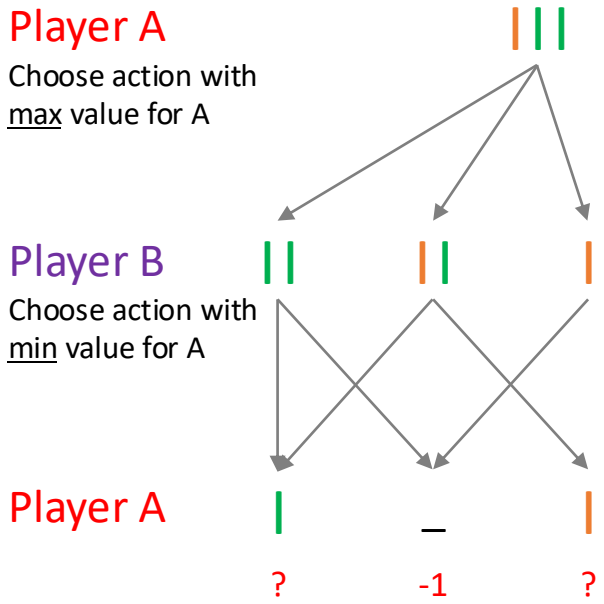
Choose action with min value for A

Minimax

```
def minimax(state):
    (action, v) = max_value(state)
    return action
```

```
def max_value(state):
    if is_terminal(state): return utility(state)
    v = -∞ // smallest value possible
    for next_state in expand(state):
        v = max(v, min_value(next_state))
    return v
```

```
def min_value(state):
    if is_terminal(state): return utility(state)
    v = ∞ // largest value possible
    for next_state in expand(state):
        v = min(v, max_value(next_state))
    return v
```



Note: The terminal state is the state "no sticks". If this state is reached:
On Player A's turn, then **utility**(state) outputs -1
On Player B's turn, then **utility**(state) outputs +1

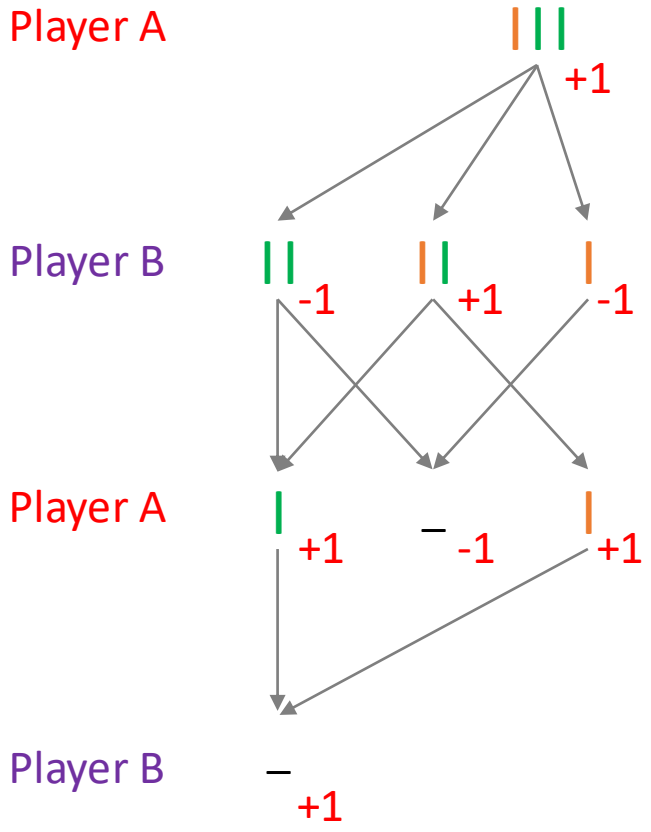
Hence the implementation **utility**(state) must ensure that the correct utility is returned in **max_value** and **min_value,** respectively.

Minimax – Demo

```
def minimax(state):
  (action, v) = max_value(state)
  return action

def max_value(state):
  if is_terminal(state): return utility(state)
  v = -∞ // smallest value possible
  for next_state in expand(state):
    v = max(v, min_value(next_state))
  return v

def min_value(state):
  if is_terminal(state): return utility(state)
  v = ∞ // largest value possible
  for next_state in expand(state):
    v = min(v, max_value(next_state))
  return v
```

Player A: maximize

Player B: minimize



Minimax – Analysis

```
def minimax(state):
  (action, v) = max_value(state)
  return action

def max_value(state):
  if is_terminal(state): return utility(state)
  v = -∞ // smallest value possible
  for next_state in expand(state):
    v = max(v, min_value(next_state))
  return v

def min_value(state):
  if is_terminal(state): return utility(state)
  v = ∞ // largest value possible
  for next_state in expand(state):
    v = min(v, max_value(next_state))
  return v
```

Time Complexity?

Exponential w.r.t. maximum depth: $O(b^m)$

Branching factor b, max-depth m

Space Complexity?

Polynomial

Complete?

Yes, if tree is finite

Optimal?

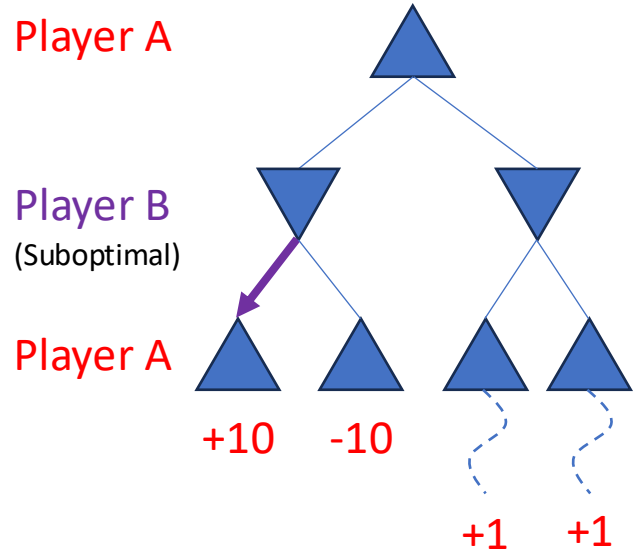
Yes, against optimal opponent

Minimax – Optimality

Theorem: In any finite, two-player, zero-sum game of perfect information, the minimax algorithm computes the **optimal strategy** for both players, guaranteeing the best achievable outcome for each, **assuming both play optimally**.

Theorem: If Player A's **opponent** deviates from optimal play (i.e., **plays sub-optimally**), then for any action taken by Player A, the **utility** obtained by Player A is **never less than** the utility they would obtain **against an optimal opponent**.

However, there may be other strategies that perform even better (e.g., end the game faster, get a better outcome) against a suboptimal opponent.

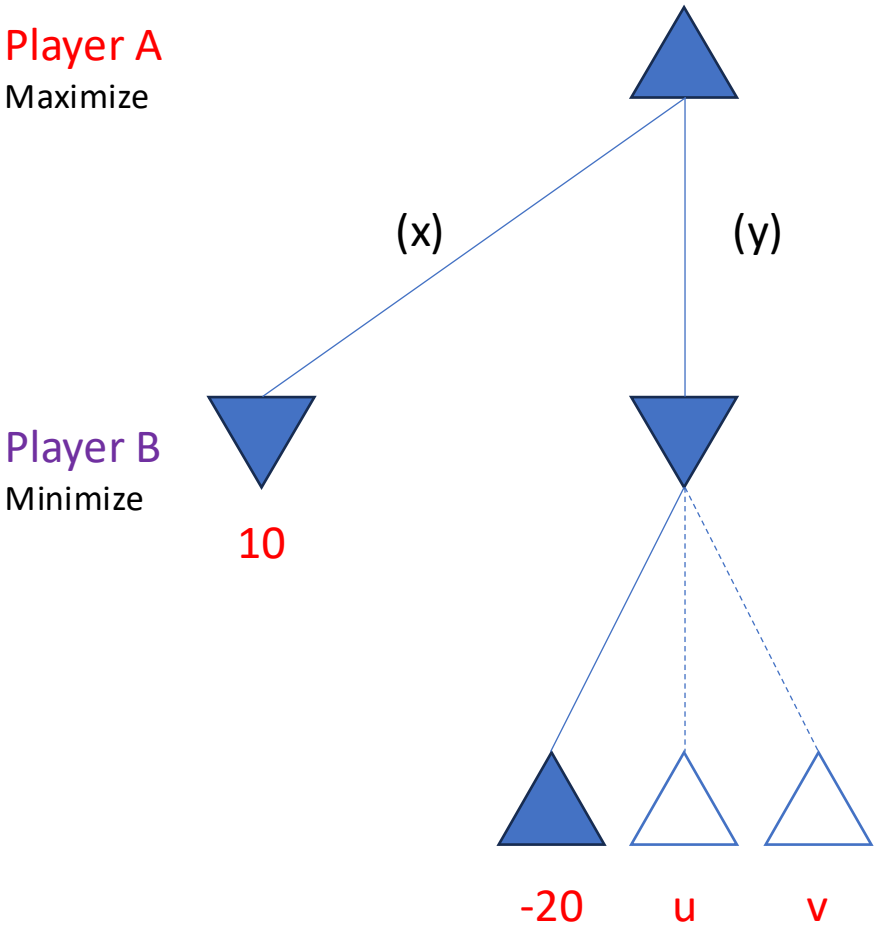


Q: Which move would you choose?

Assume you are Player A.

Opponent plays optimally.

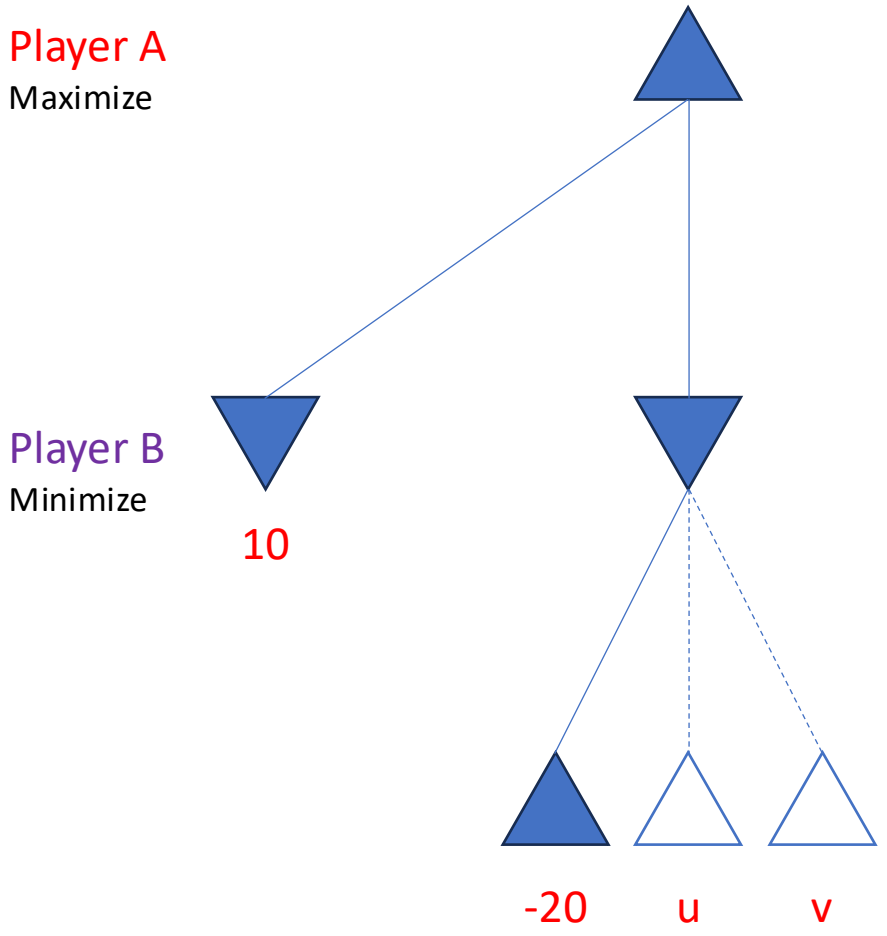
Do you choose move (x) or (y)?



The Problem with Minimax

Minimax explores the entire game tree.

However, evaluating a node sometimes becomes unnecessary, as it won't alter the decision.



Logical view:

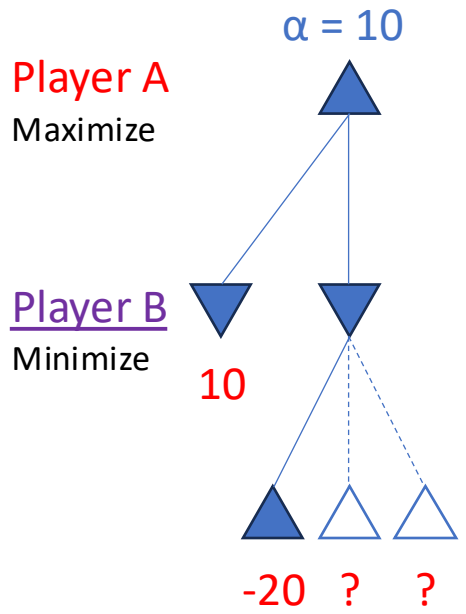
max(10, min(-20,u,v)) = 10

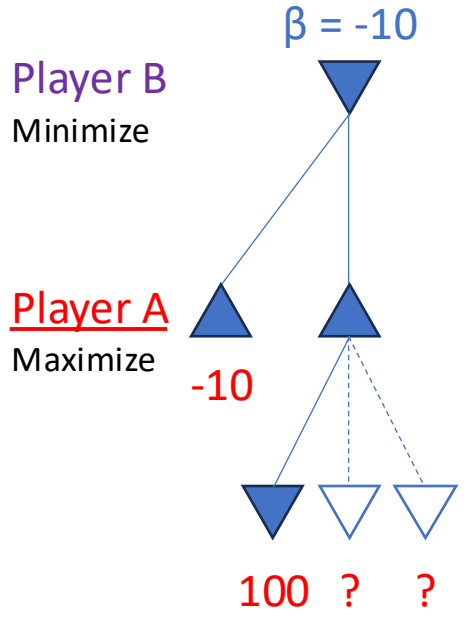
for all u,v.

Alpha-Beta Pruning – Overview

A variant of Minimax algorithm that seeks to decrease the number of nodes that are evaluated.

- Maintain the best outcomes so far for Player A and Player B
  - Player A: highest value for A so far (called alpha)
  - Player B: lowest value for A so far (called beta)
- Prune unnecessary nodes
  - At the turn of Player B, if an action leads to a worse outcome for **player A** (smaller than alpha), then the rest of the actions are irrelevant; the node will have at most that outcome which Player A is not interested in.
  - At the turn of Player A, if an action leads to a better outcome for **player A** (higher than beta), then the rest of the actions are irrelevant; the node will have at least that outcome which Player B is not interested in.



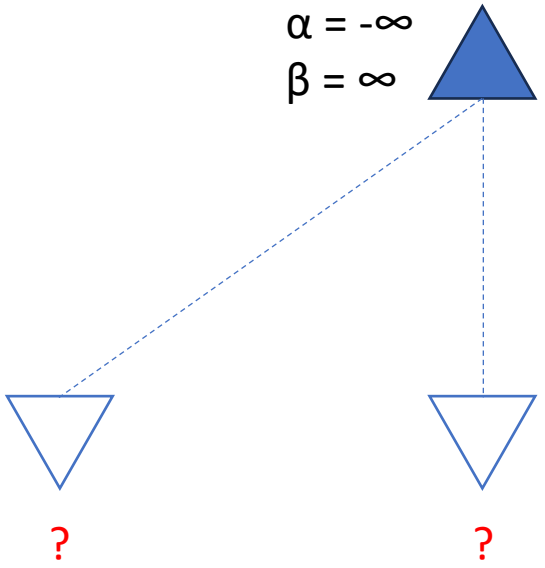


```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action
```

Alpha-Beta Pruning

Player A

Choose action with max value for A



Best outcome so far

Player A: highest value for A so far (α)

Player B: lowest value for A so far (β)

At the beginning, both players are **pessimistic** about the best outcome:

- Worst-case for A: -∞ (A loses big)
- Worst-case for B: ∞ (A wins big)

```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v
```

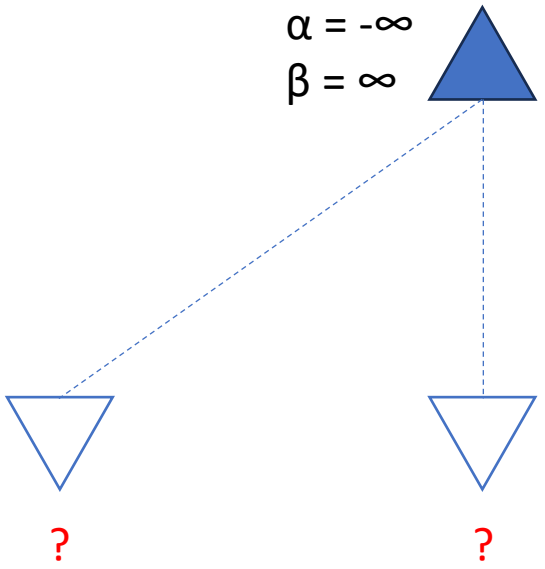
Alpha-Beta Pruning

Player A

Choose action with max value for A

Player B

Choose action with min value for A



Best outcome so far

Player A: highest value for A so far (α)

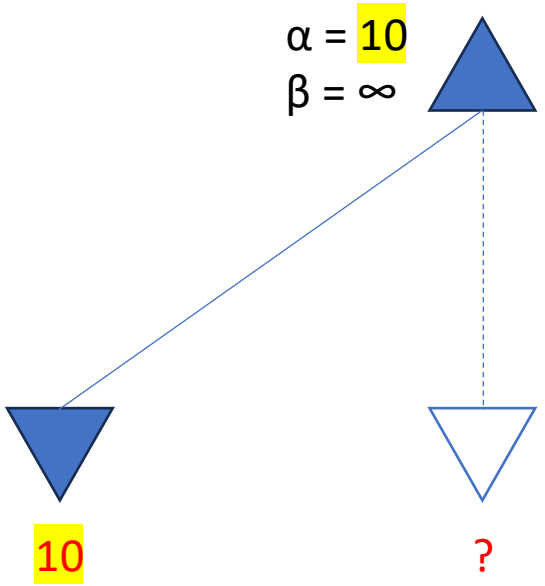
Player B: lowest value for A so far (β)

```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v
```

Alpha-Beta Pruning

Player A
Choose action with max value for A

Player B
Choose action with min value for A



Best outcome so far
Player A: highest value for A so far (α)
Player B: lowest value for A so far (β)

Outcome for A in this node is better than best so far (-∞), update α

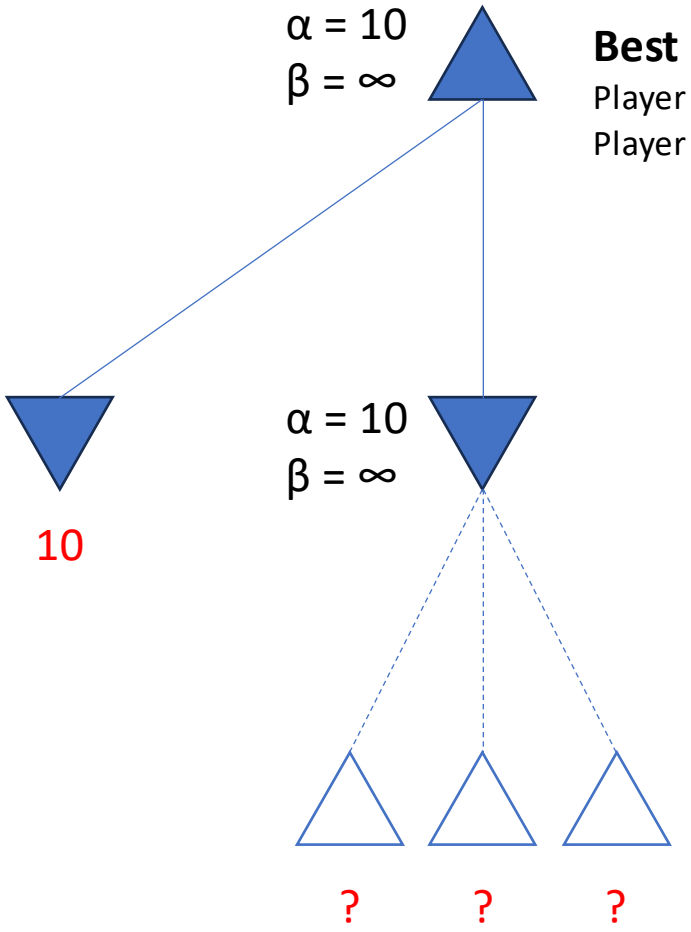
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```

Alpha-Beta Pruning

Player A
Choose action with max value for A

Player B
Choose action with min value for A

Player A



Best outcome so far
Player A: highest value for A so far (α)
Player B: lowest value for A so far (β)

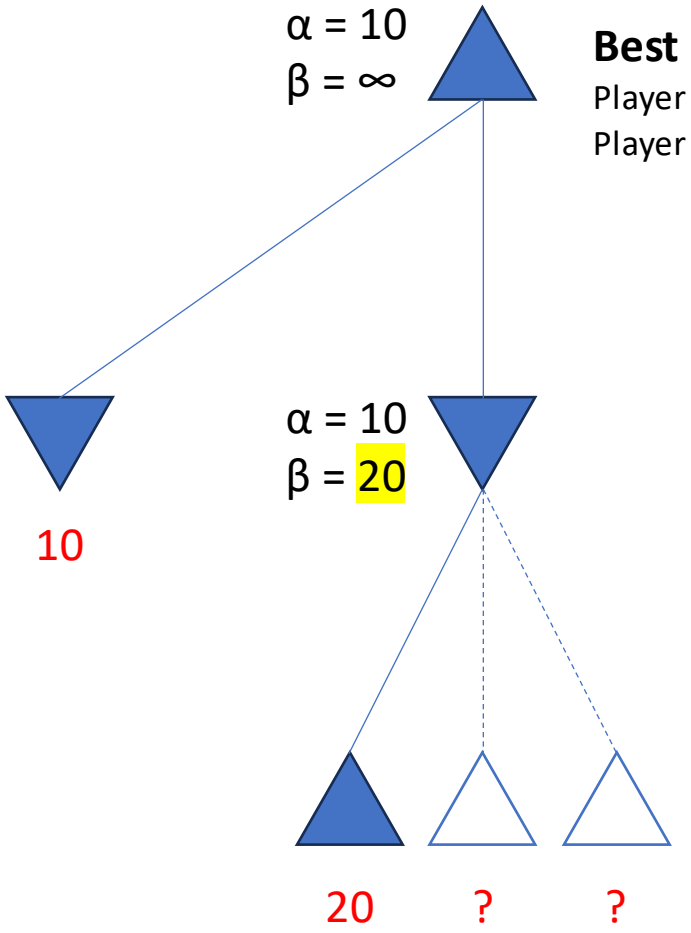
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```

Alpha-Beta Pruning

Player A
Choose action with max value for A

Player B
Choose action with min value for A

Player A



Best outcome so far
Player A: highest value for A so far (α)
Player B: lowest value for A so far (β)

Outcome for B in this node is better than best so far (∞), update β

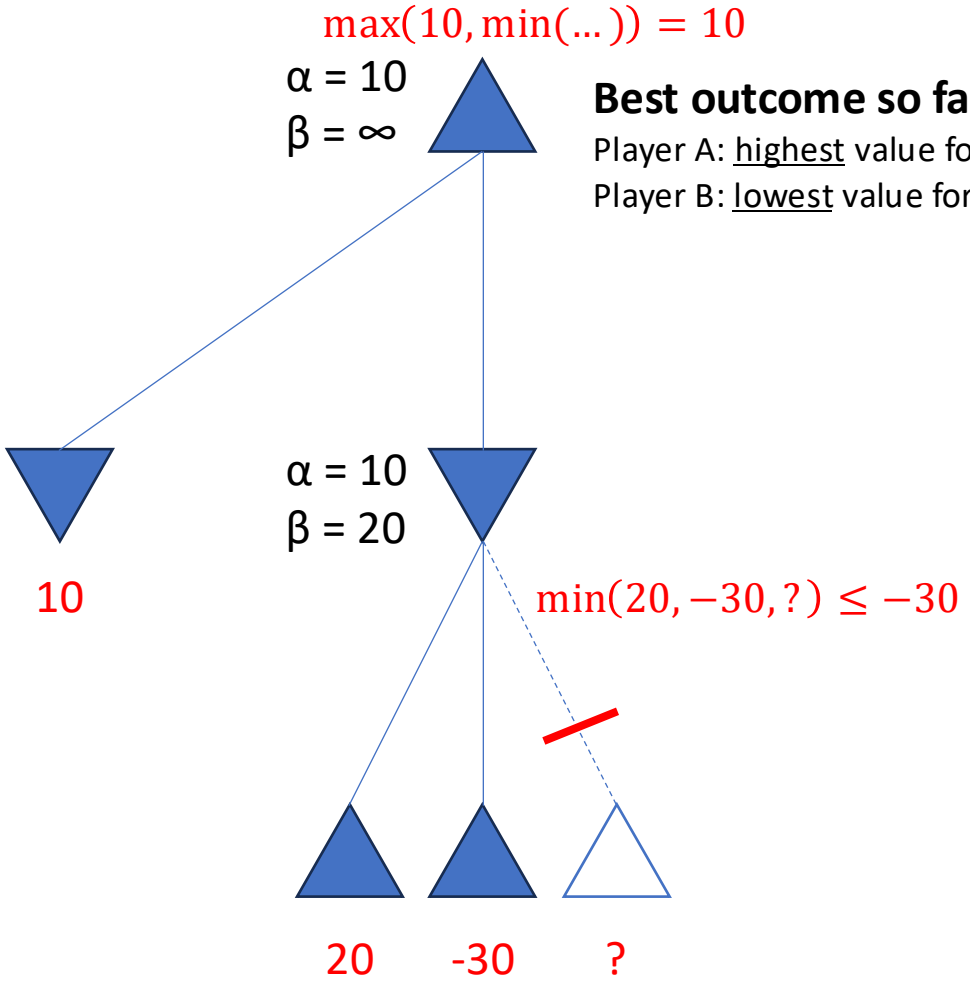
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```

Alpha-Beta Pruning

Player A
Choose action with max value for A

Player B
Choose action with min value for A

Player A



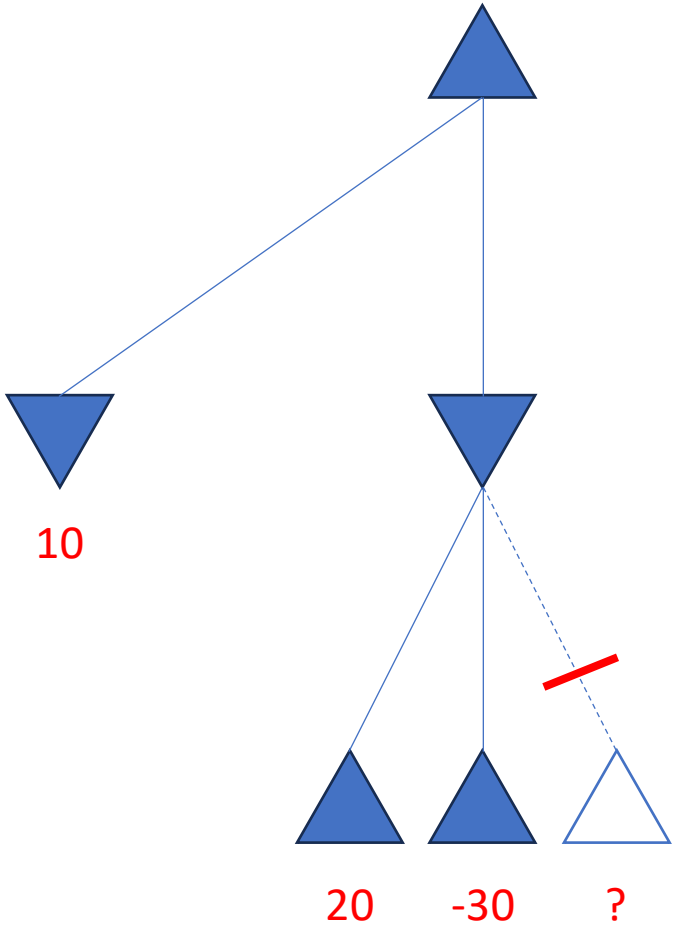
Best outcome so far
Player A: highest value for A so far (α)
Player B: lowest value for A so far (β)

Worse than the best outcome for Player A so far (10)
Player A in the root node will not pick the 2nd action

Alpha-Beta Pruning – Analysis

Theorem: Alpha-beta pruning **does not alter** the minimax **value** and the **optimal move** computed for the root node; it only reduces the number of nodes evaluated.

Logical view: $\max(10, \min(20, -30, ?)) = 10$

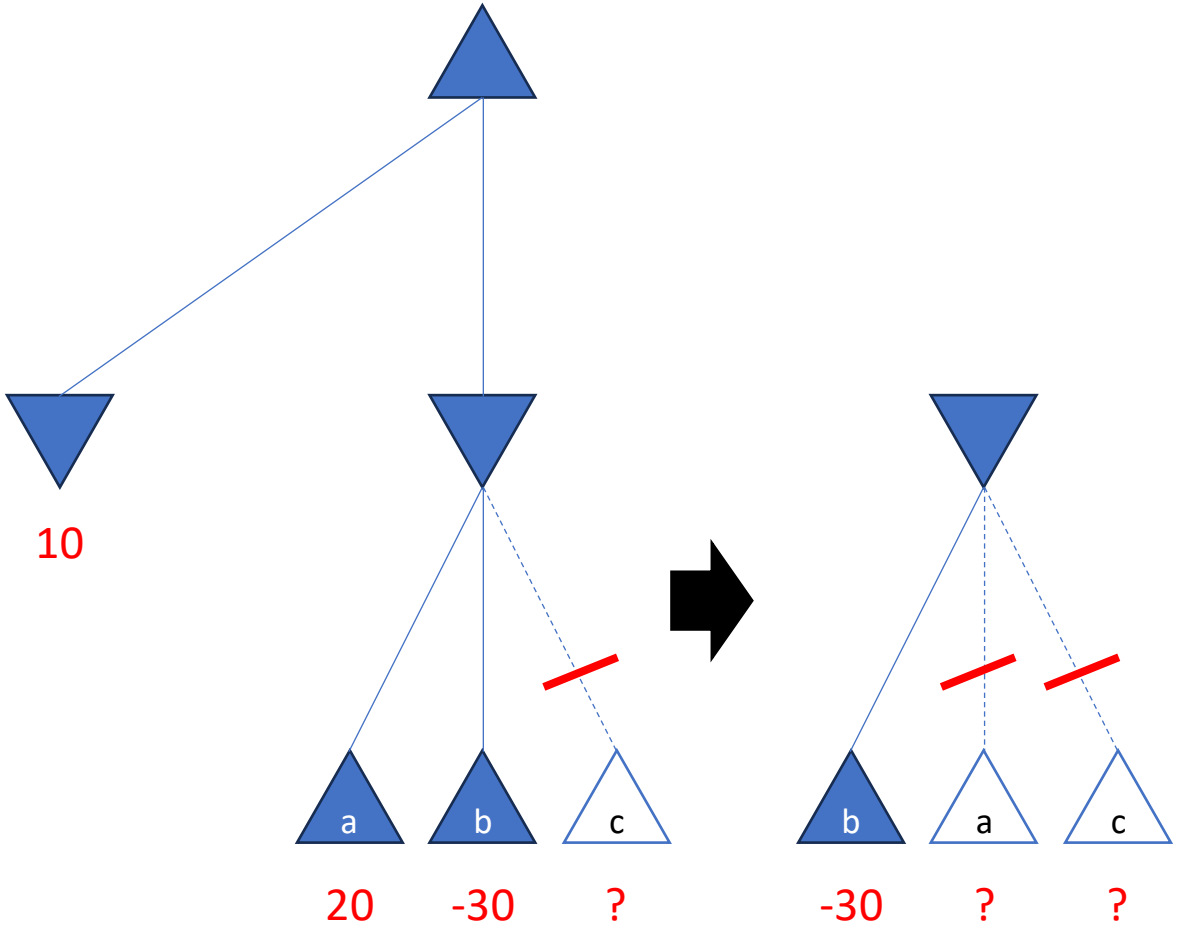


Alpha-Beta Pruning – Analysis

Theorem: Alpha-beta pruning **does not alter** the minimax **value** and the **optimal move** computed for the root node; it only reduces the number of nodes evaluated.

Logical view: $\max(10, \min(20, -30, ?)) = 10$

Making full use of alpha-beta pruning requires **meta-reasoning**: reasoning about which computations to do first.



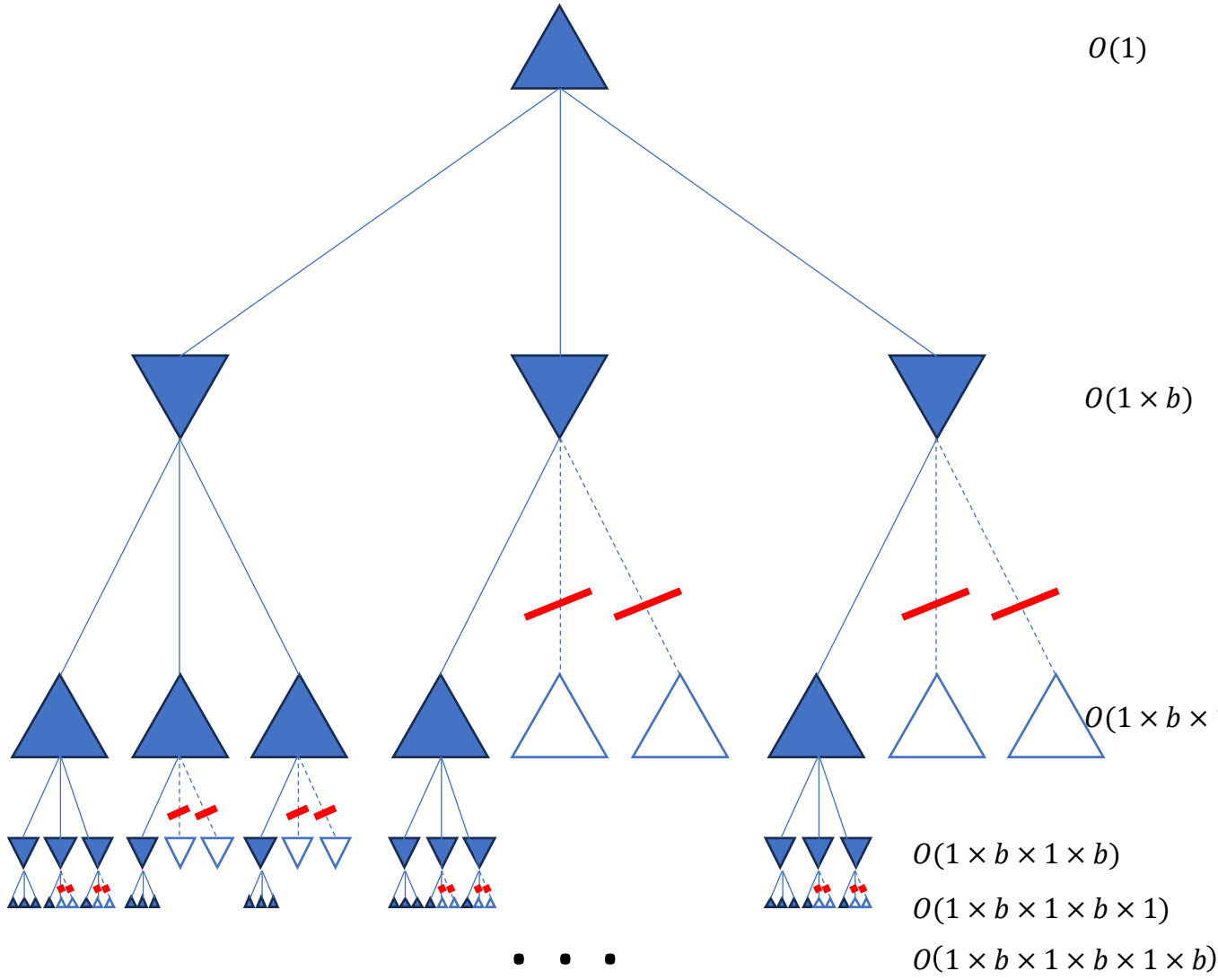
Alpha-Beta Pruning – Analysis

Theorem: Alpha-beta pruning **does not alter** the minimax **value** and the **optimal move** computed for the root node; it only reduces the number of nodes evaluated.

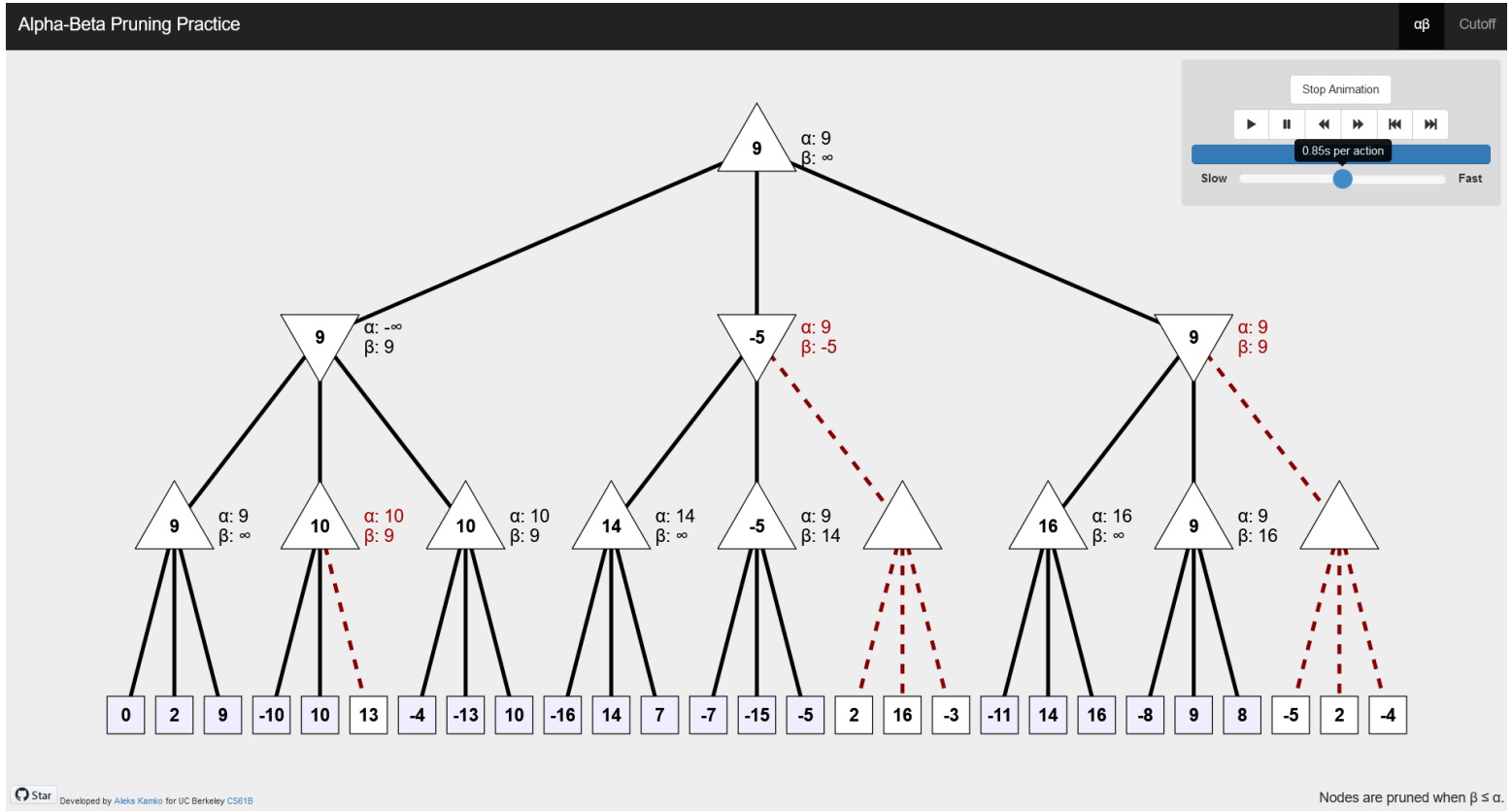
Logical view: $\max(10, \min(20, -30, ?)) = 10$

Making full use of alpha-beta pruning requires **meta-reasoning**: reasoning about which computations to do first.

Theorem: Good move ordering increases alpha-beta pruning effectiveness; with "perfect ordering", its complexity is $O\left(b^{\frac{m}{2}}\right)$



Resources

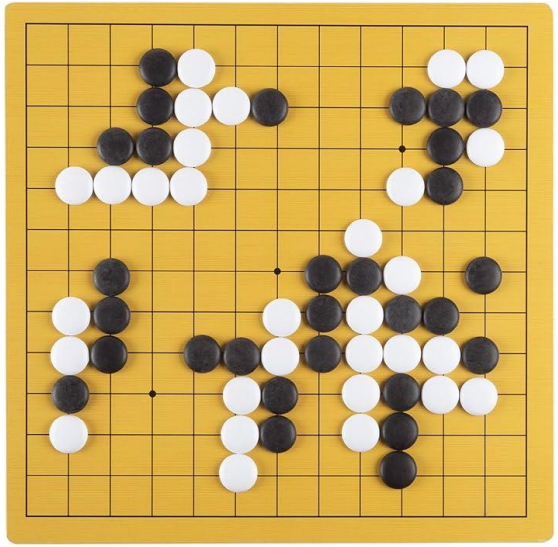


https://pascscha.ch/info2/abTreePractice/

Note: can't be accessed from NUS network

“Real-world” Games

Credit: chess.com



Credit: Amazon.sg

$b \approx 35, m \approx 80$ for “reasonable” games

Time complexity: $b^m = 35^{80} \approx 10^{123}$

$b \approx 250, m \approx 150$ for “reasonable” games

Time complexity: $b^m = 250^{150} \approx 10^{360}$

Number of particles in the universe: $\sim 10^{80}$

Imperfect Decisions by Using Cutoff

In real-world games, calculating the utility of a state by fully exploring the entire game tree with Minimax is often computationally infeasible (even with alpha-beta pruning).

Instead of evaluating the full tree,

- We can apply a **cutoff** strategy, halting the search in the middle, and
- Estimate the value of (mid-game) states using an **evaluation function**.

The cutoff is typically based on constraints such as the available computational budget (e.g., **time** or **search depth**).

Use function **is_cutoff**(state)

- If **is_terminal**(state) = True, then return True.
- If cutoff criterion is satisfied, then return True.
- Otherwise, return False.

Evaluation Function

- **Evaluation function** is a function that assigns a score to a game state, indicating how favorable it is for a player.
  - If a terminal state: use utility(state)
  - Otherwise: use a heuristic
- **Heuristic function** is a function that *estimates* the utility of a state.
  - A heuristic should fall somewhere between the minimum and maximum utility, such as between a loss and a win.
    - i.e., $\min_s utility(s) \leq heuristic(state) \leq \max_s utility(s)$, for all states
    - Example: $utility(loss) \leq heuristic(state) \leq utility(win)$, for all states
  - Computation must not be too expensive
  - In adversarial search, admissibility and consistency properties do not apply

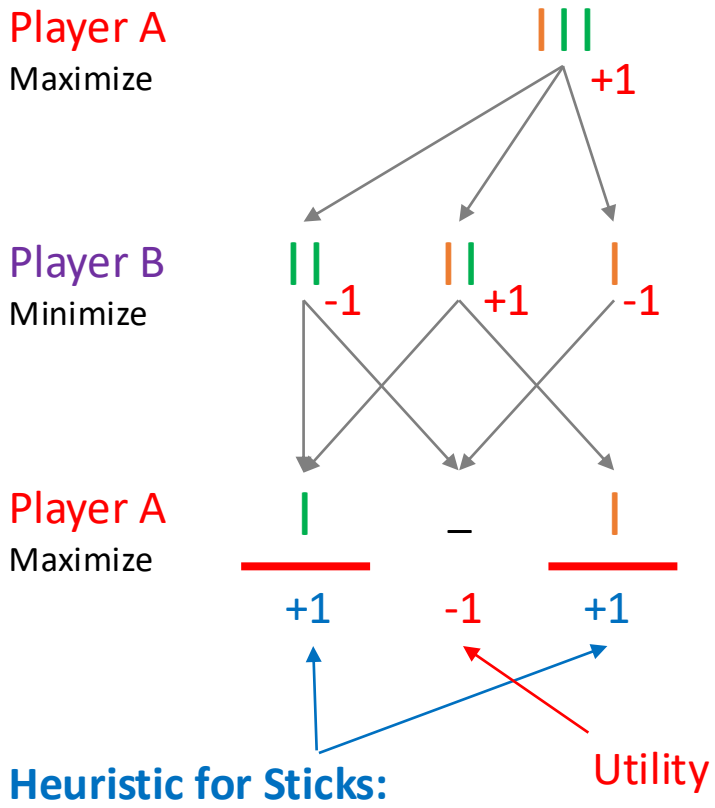
“Inventing” a Heuristic

- Relaxed game – in principle, but usually too costly in practice
  - **Relaxed Chess**: each piece can teleport and perform self-sacrificial attack
    - Heuristic: number of agent’s pieces – number of opponent’s pieces
- Expert knowledge
  - **Chess**: often uses features like material count, mobility, safety, etc.
- Learn from data
  - Heuristic function could be learned from labeled (state, utility) dataset. The dataset could be generated using some expert agent or by solving the game (or some portions of the game)

Minimax with Cutoff

```
def minimax_with_cutoff(state):
    (action, v) = max_value(state)
    return action

def max_value(state):
    if is_cutoff(state): return eval(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state))
    return v

def min_value(state):
    if is_cutoff(state): return eval(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state))
    return v
```

Max depth = 2



Heuristic for Sticks:

+1 If the number of sticks ≥ 1

-1 otherwise

Minimax with Cutoff – Optimality

Theorem: In a finite, two-player, zero-sum game, the minimax algorithm with cutoff and an evaluation function returns a move that is **optimal with respect to the evaluation function** at the cutoff.

This move may be **suboptimal with respect to the true minimax value.**

Summary

- Local search
  - Problems with large state space, good enough solution is okay
  - Hill climbing: pick the best the neighbour, repeat
  - State space: local/global minima, shoulder
- Adversarial search
  - Competitive two-player turn-taking zero-sum games
  - Minimax: max of min value
  - Alpha-beta pruning: prune branches that don’t affect decision
  - Handling large game trees: cutoff, evaluation/heuristic function

Further Reading (Optional)

- Local Search
  - Simulated annealing (AIAMA 4.1.2)
  - Escape techniques
    - Random restarts (AIAMA 4.1.1)
    - Tabu search (AIAMA Ch. 4, pg. 154)
  - Local beam search (AIAMA 4.1.3)
  - Genetic algorithm (AIAMA 4.1.4)
- Adversarial Search
  - Move ordering in alpha-beta pruning (AIAMA 5.2.4)
  - Optimizing the search with lookup (AIAMA 5.3.4)
  - Monte-Carlo tree search (AIAMA 5.4)
  - AlphaGo (Mastering the game of Go without human knowledge, Silver et al, 2017)

Coming Up Next Week

- **Intro to Machine Learning**
- **Decision Trees**
  - Decision Tree Learning
  - Entropy and Information Gain
  - Different types of attributes
  - Pruning

To Do

- **Lecture Training 3**
  - +550 EXP
  - +100 Early bird bonus
- **Tutorial starts this week!**

Appendix

Alpha-beta Pruning

α = highest value for MAX
β = lowest value for MAX

```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action


def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v


def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```

Alpha-beta Pruning

α = highest value for MAX
β = lowest value for MAX

```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```

MAX

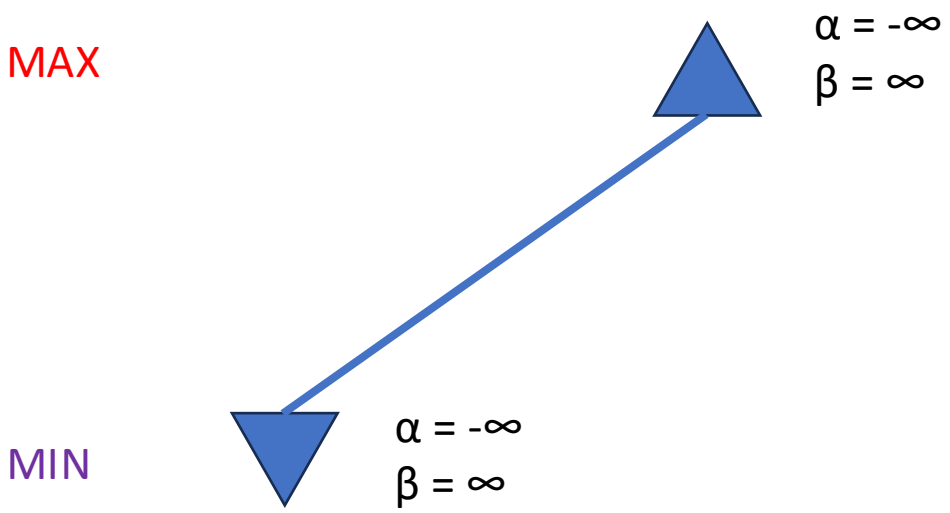
MIN

α = -∞
β = ∞

Alpha-beta Pruning

α = highest value for MAX
β = lowest value for MAX

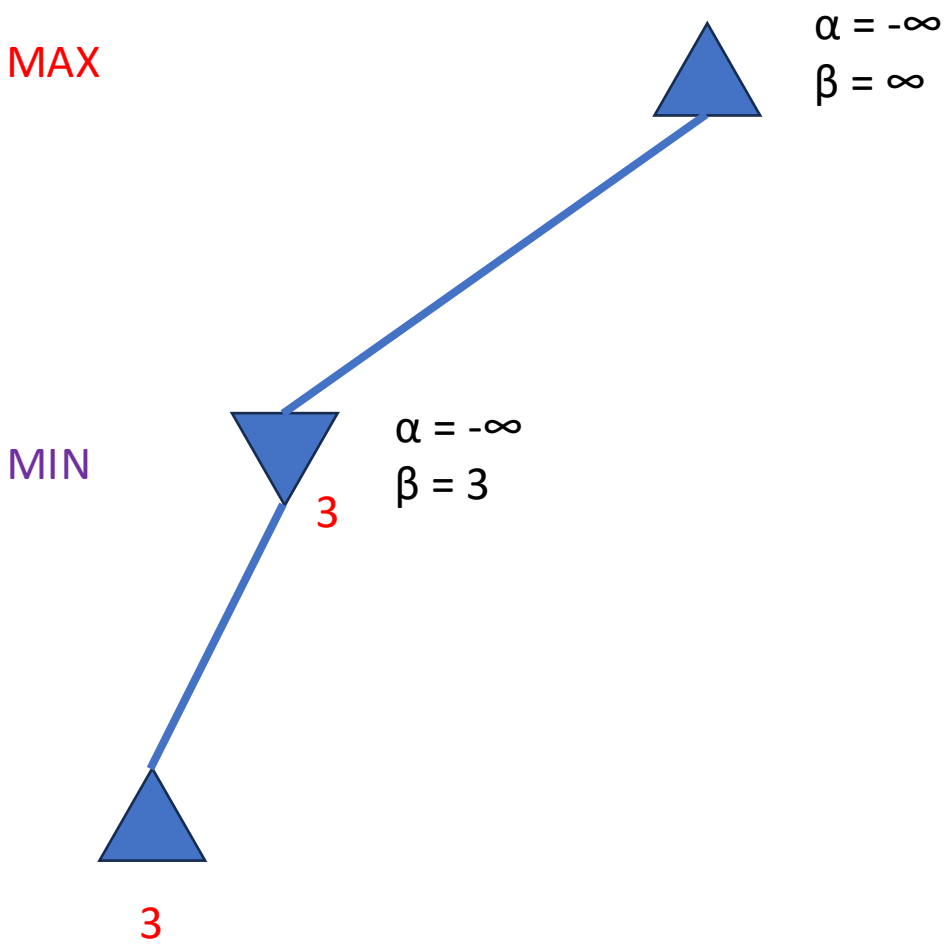
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```



Alpha-beta Pruning

α = highest value for MAX
β = lowest value for MAX

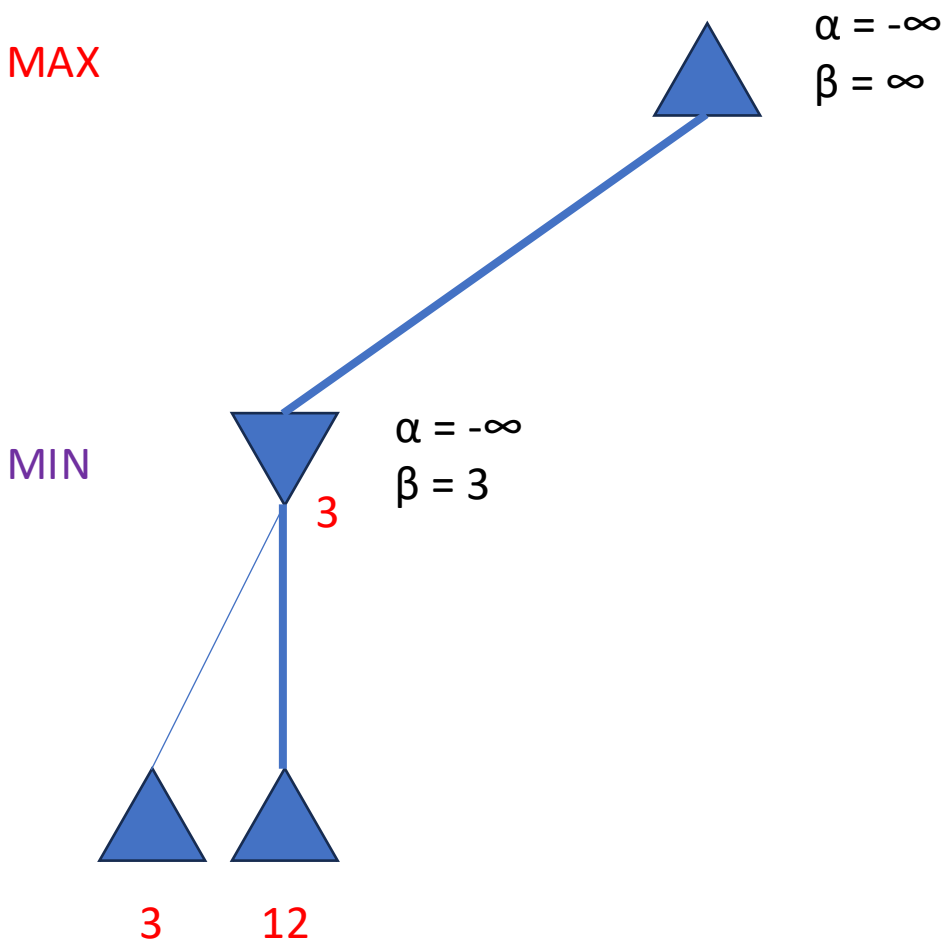
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action


def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v


def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```



Alpha-beta Pruning

α = highest value for MAX
β = lowest value for MAX

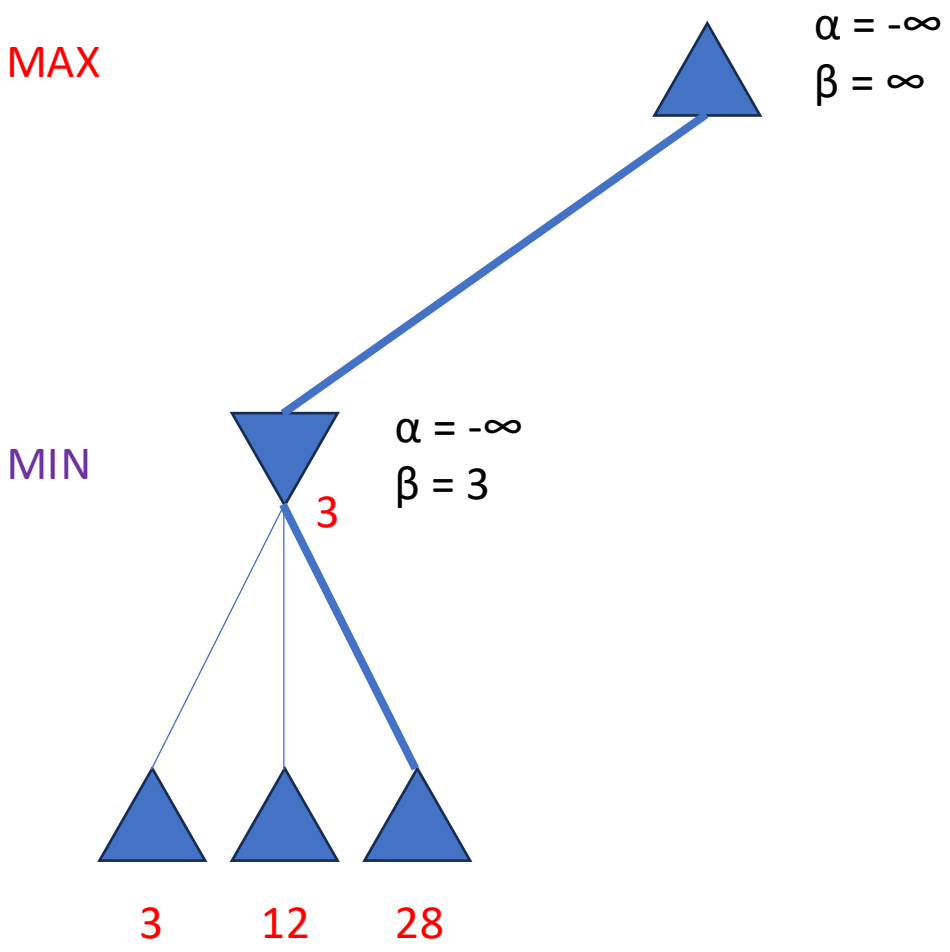
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action


def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v


def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```



Alpha-beta Pruning

α = highest value for MAX
β = lowest value for MAX

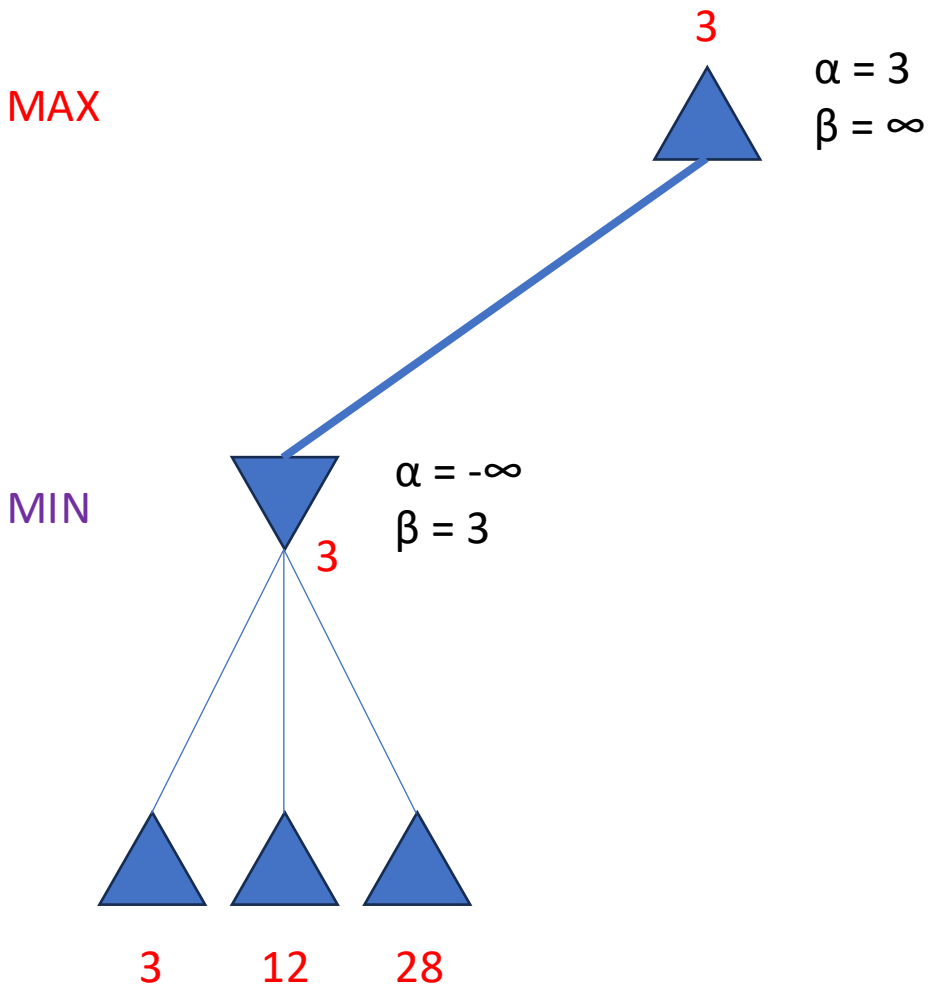
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action


def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v


def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```



Alpha-beta Pruning

α = highest value for MAX
β = lowest value for MAX

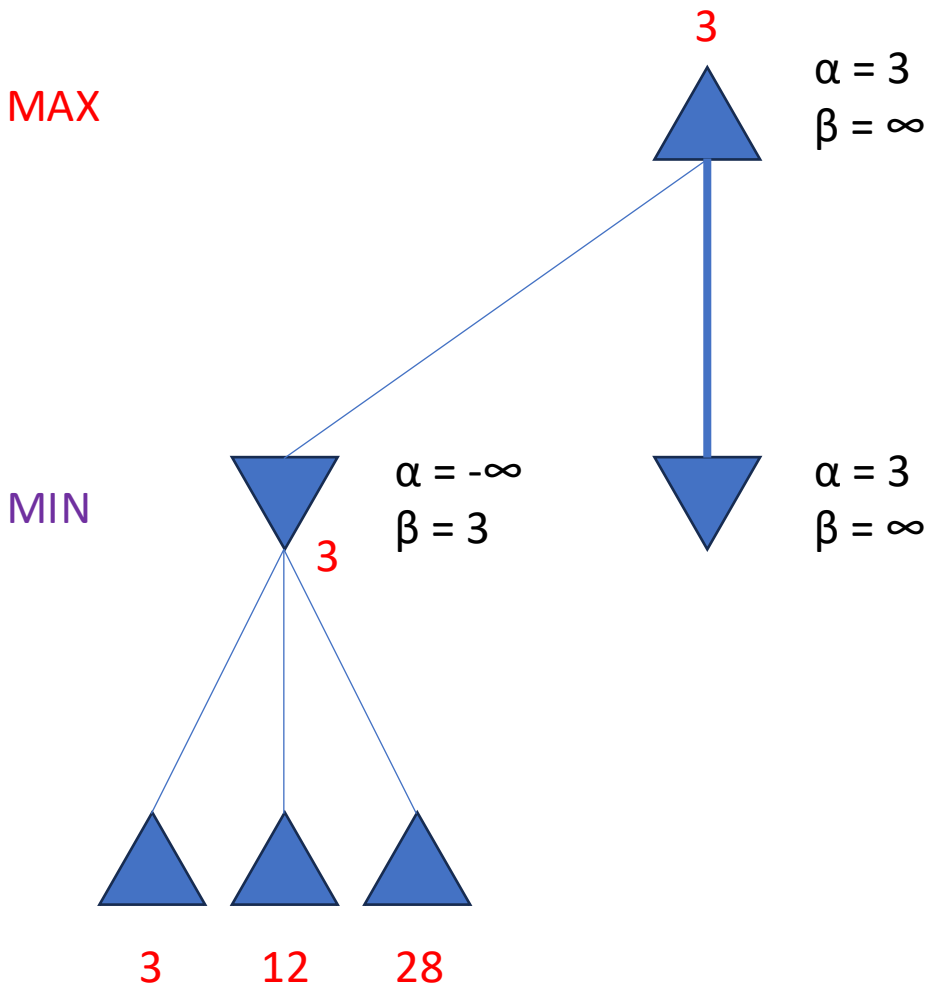
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action


def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v


def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```



Alpha-beta Pruning

α = highest value for MAX
β = lowest value for MAX

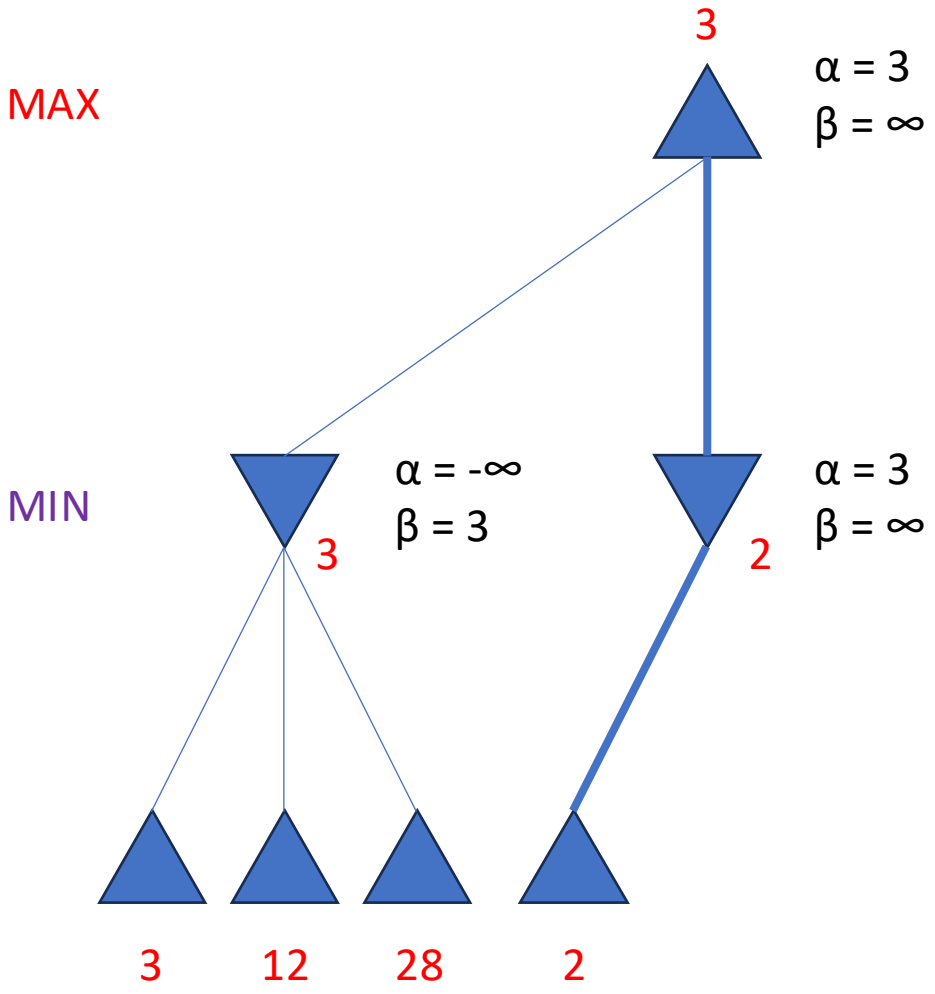
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```



Alpha-beta Pruning

α = highest value for MAX
β = lowest value for MAX

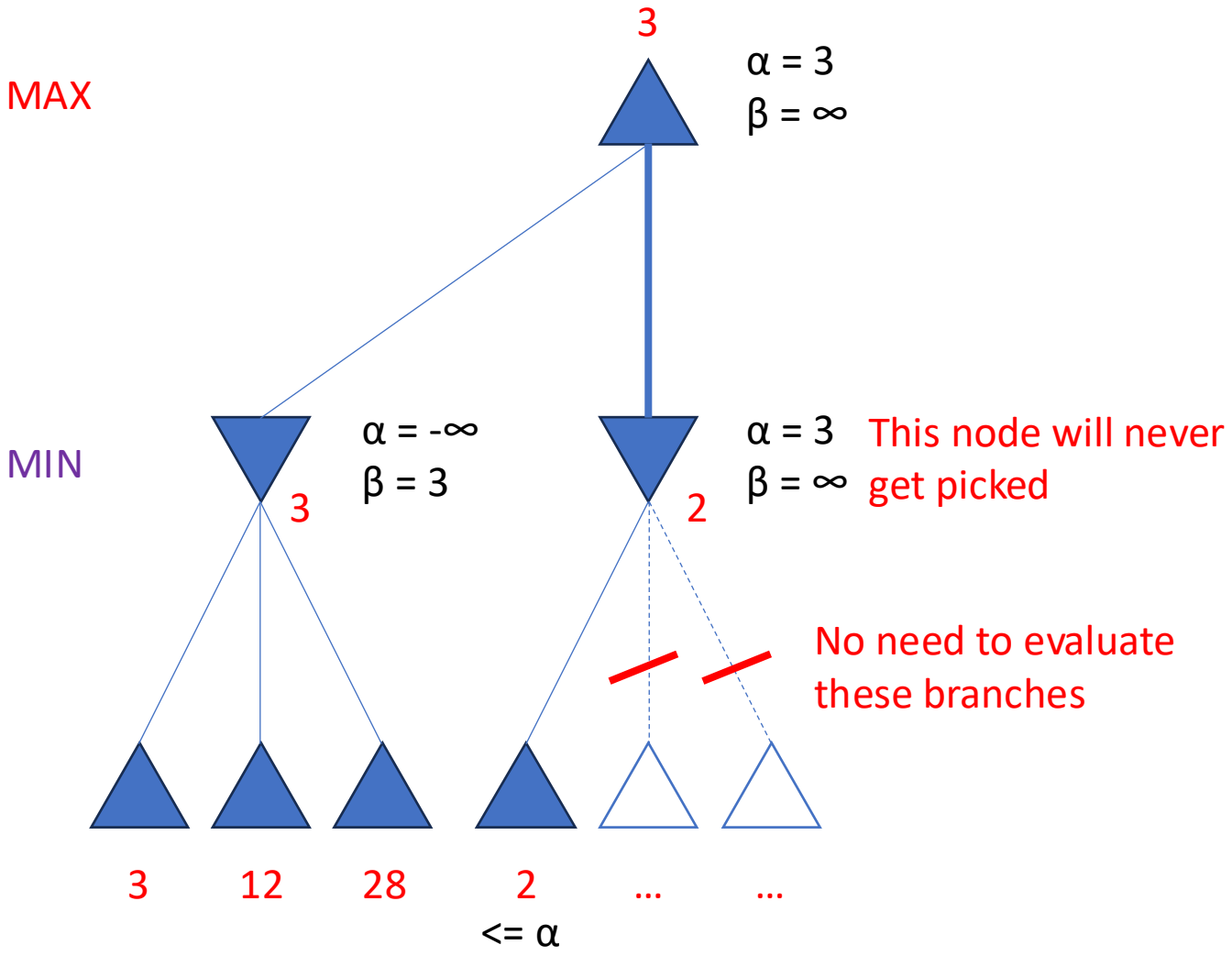
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```



Alpha-beta Pruning

```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```

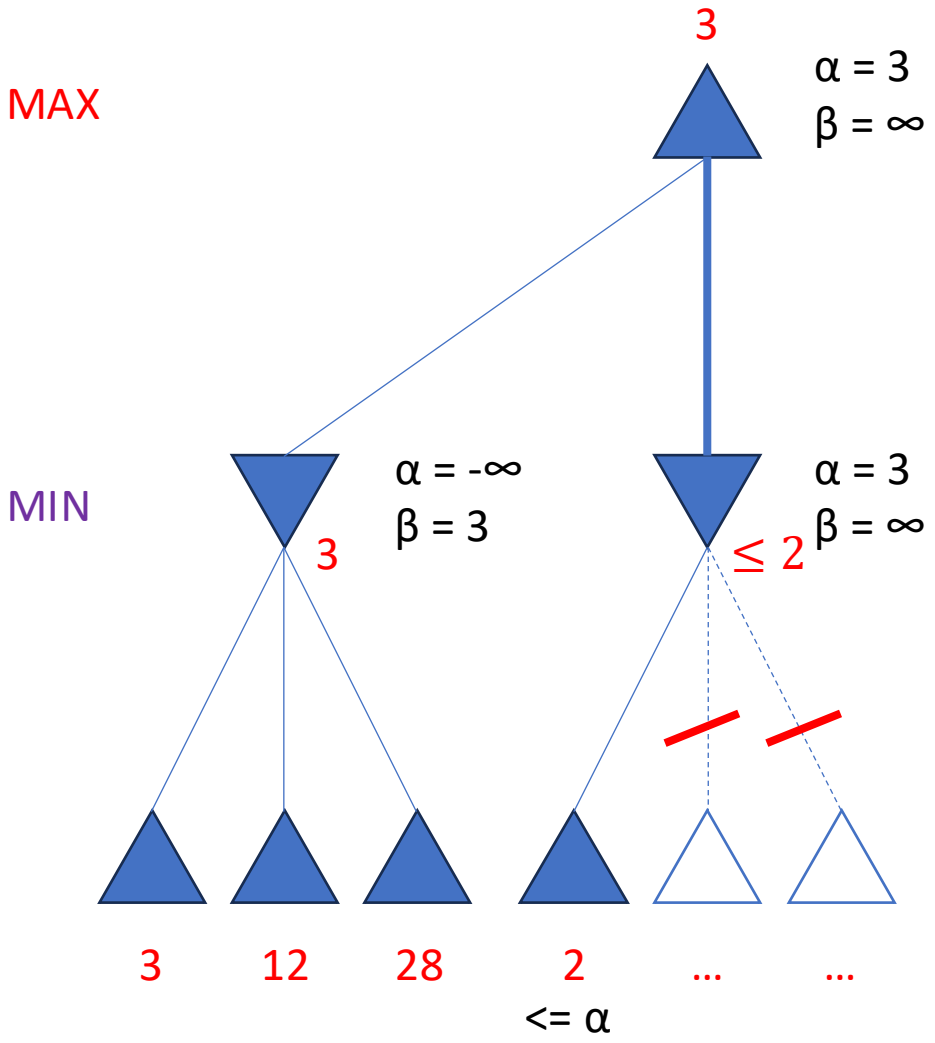
α = highest value for MAX
β = lowest value for MAX



Alpha-beta Pruning

α = highest value for MAX
β = lowest value for MAX

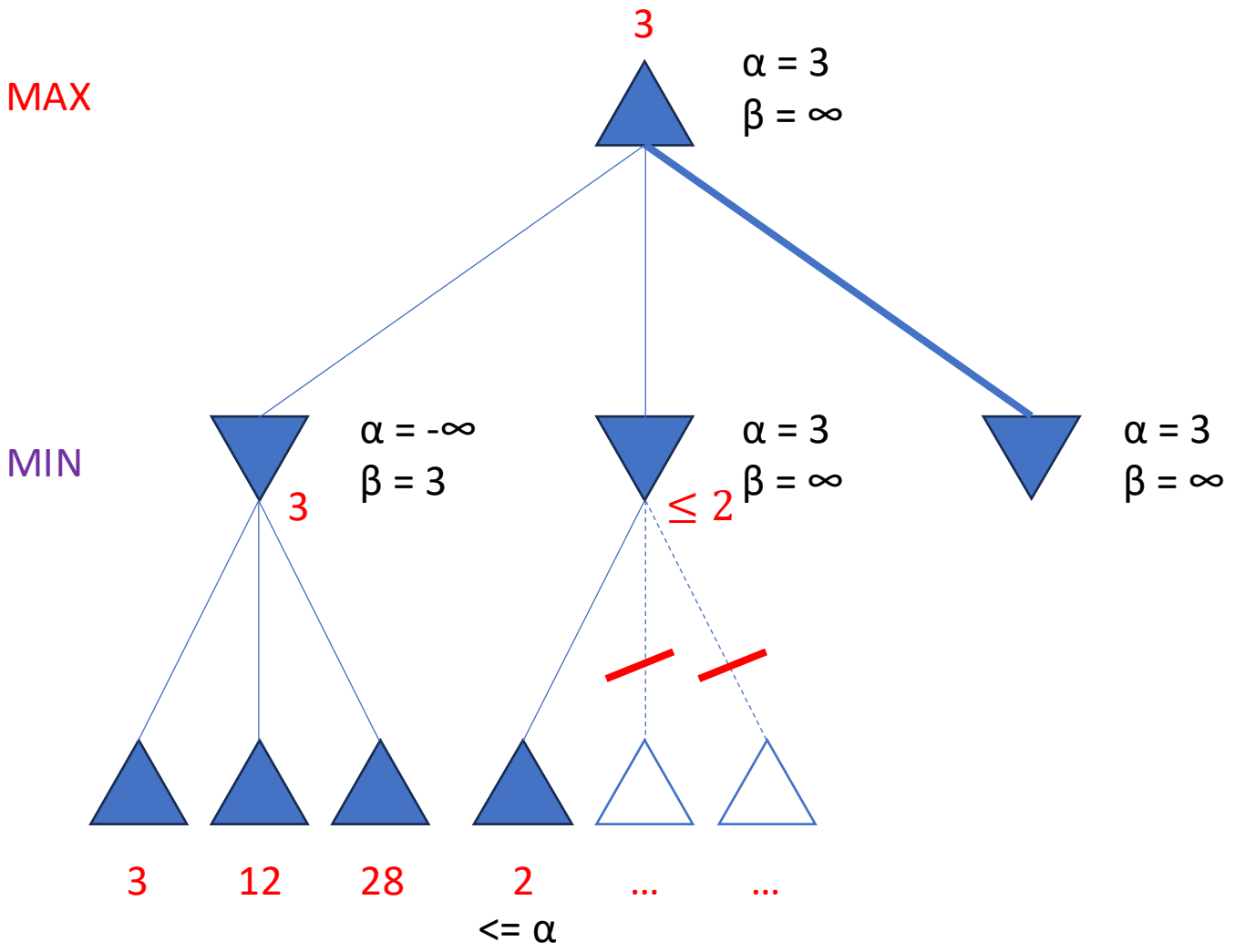
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```



Alpha-beta Pruning

α = highest value for MAX
β = lowest value for MAX

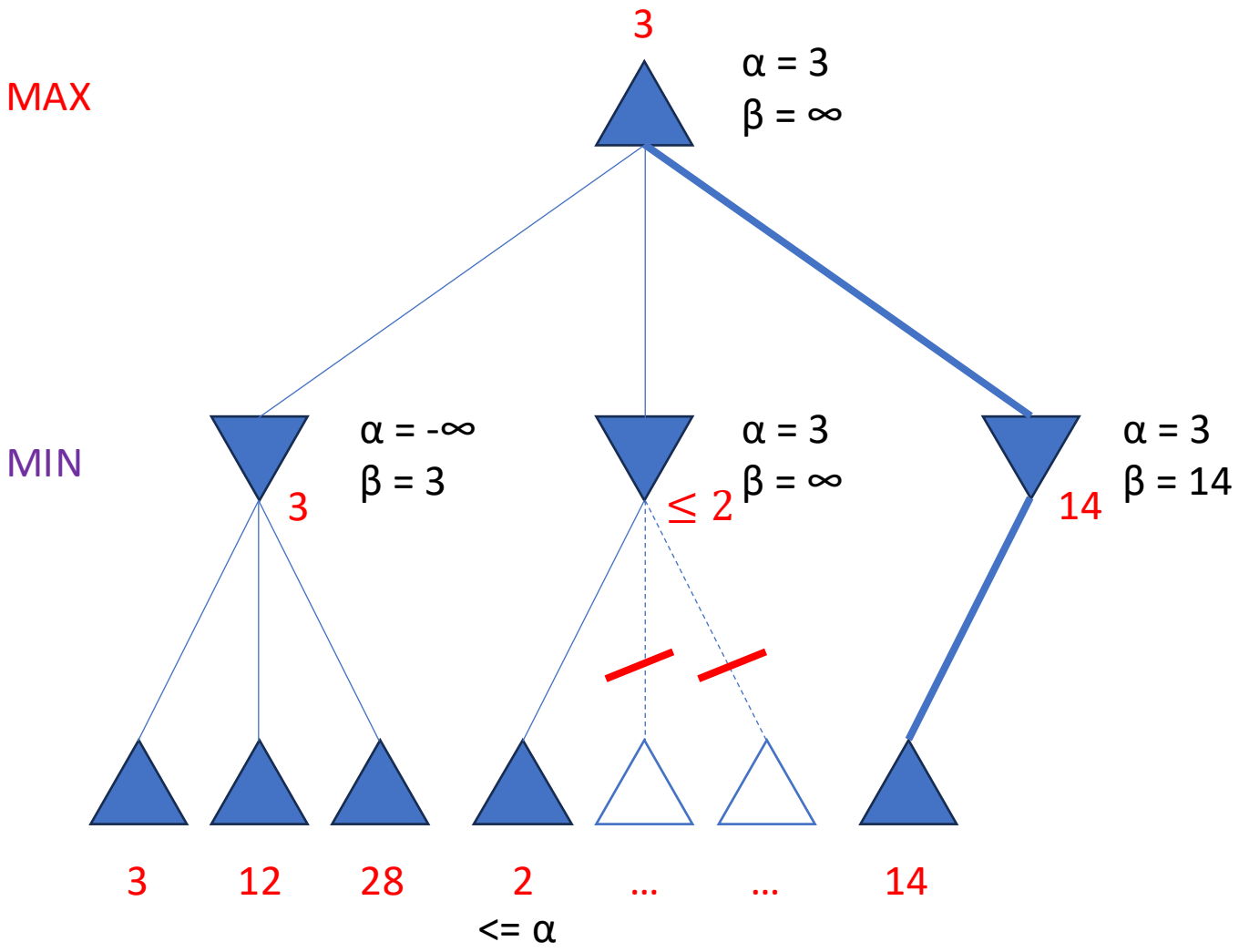
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```



Alpha-beta Pruning

α = highest value for MAX
β = lowest value for MAX

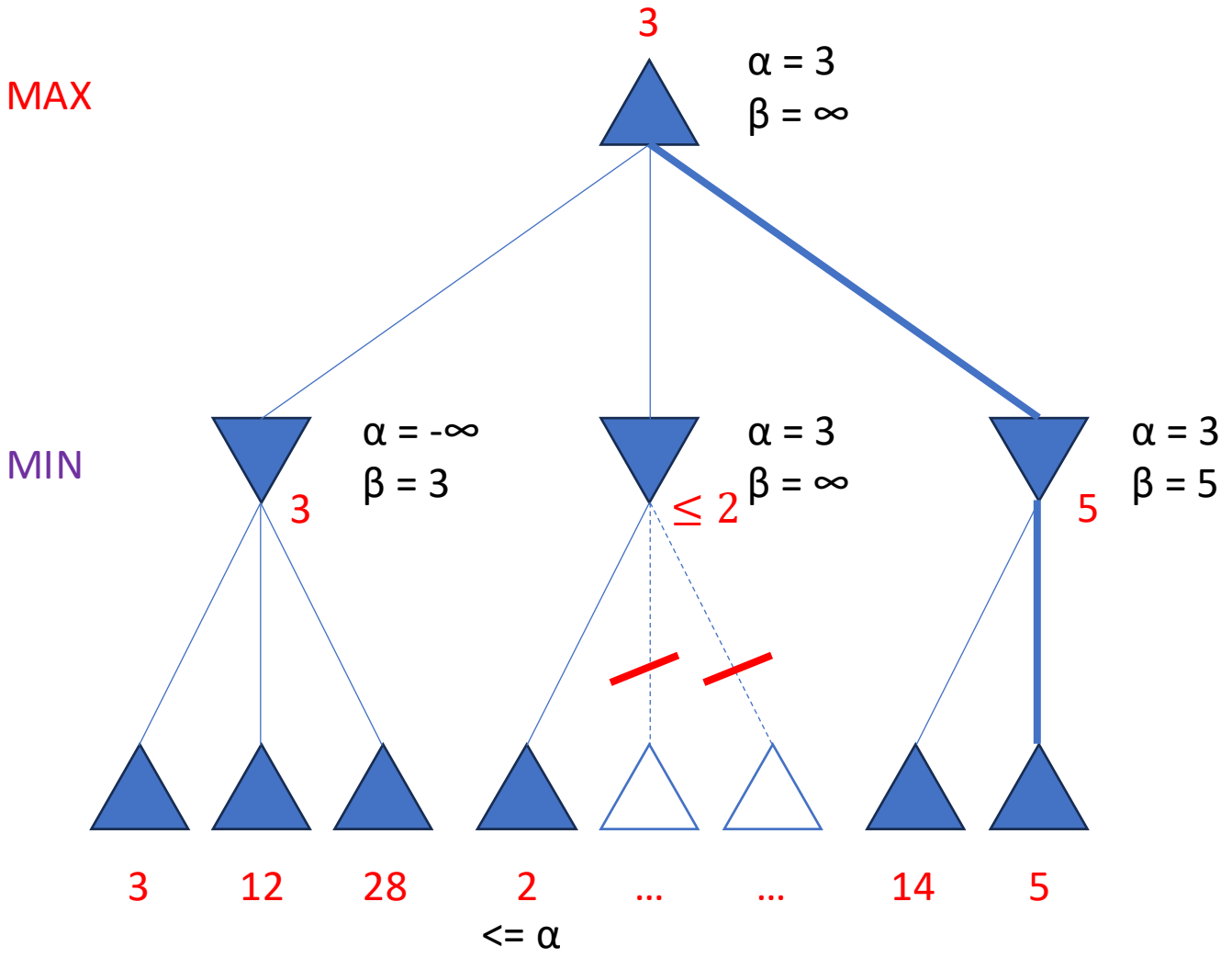
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```



Alpha-beta Pruning

α = highest value for MAX
β = lowest value for MAX

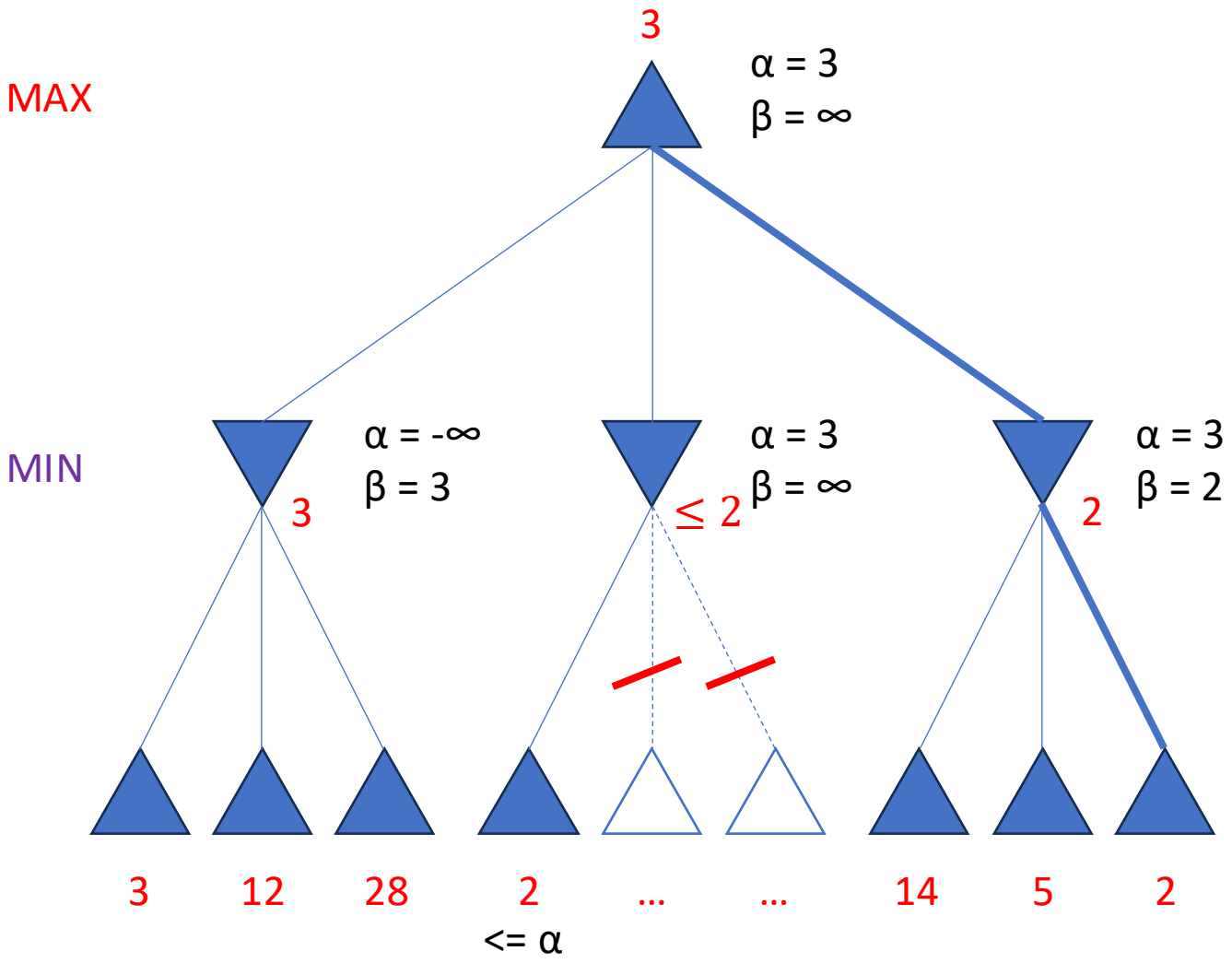
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```



Alpha-beta Pruning

α = highest value for MAX
β = lowest value for MAX

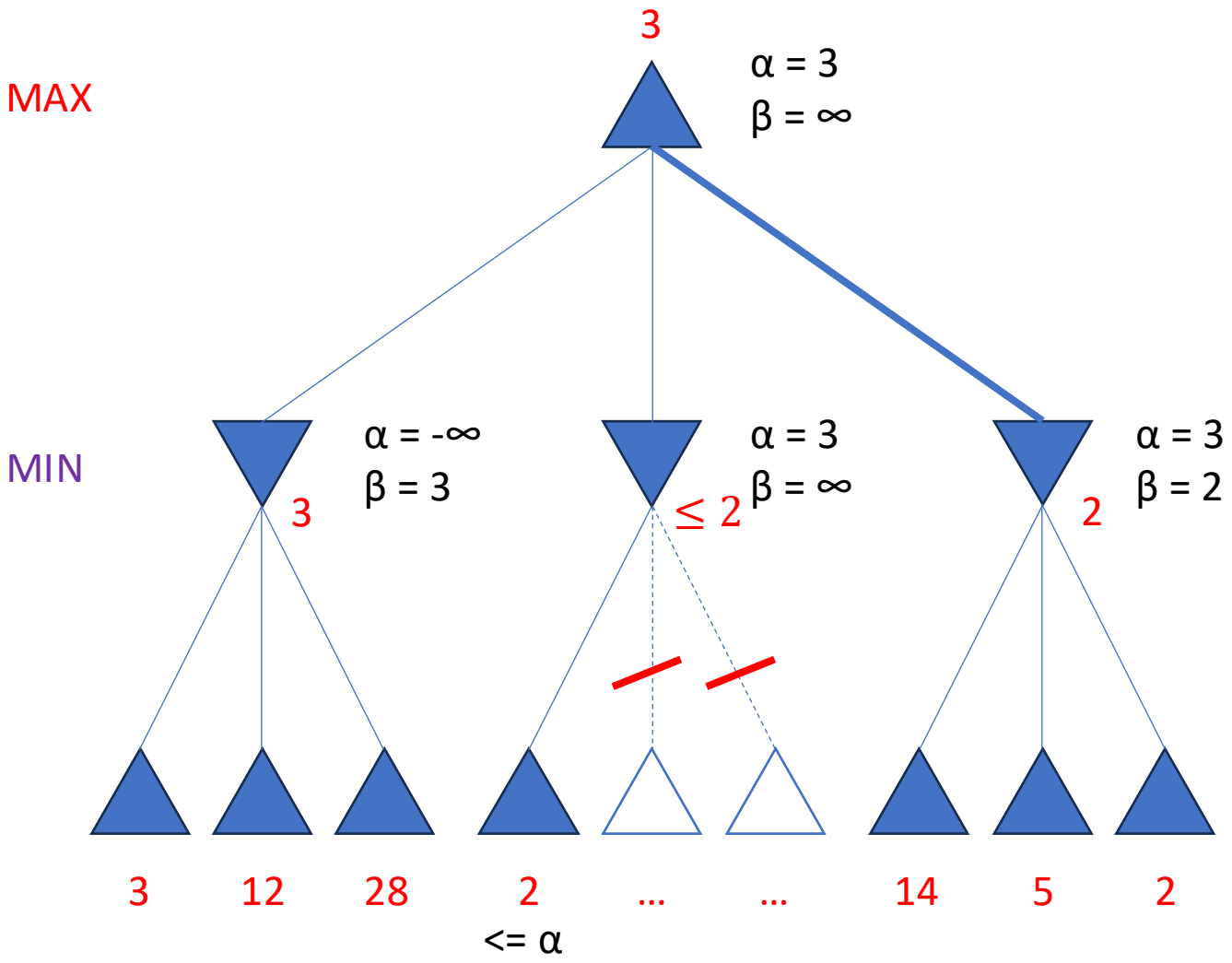
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```



Alpha-beta Pruning

α = highest value for MAX
β = lowest value for MAX

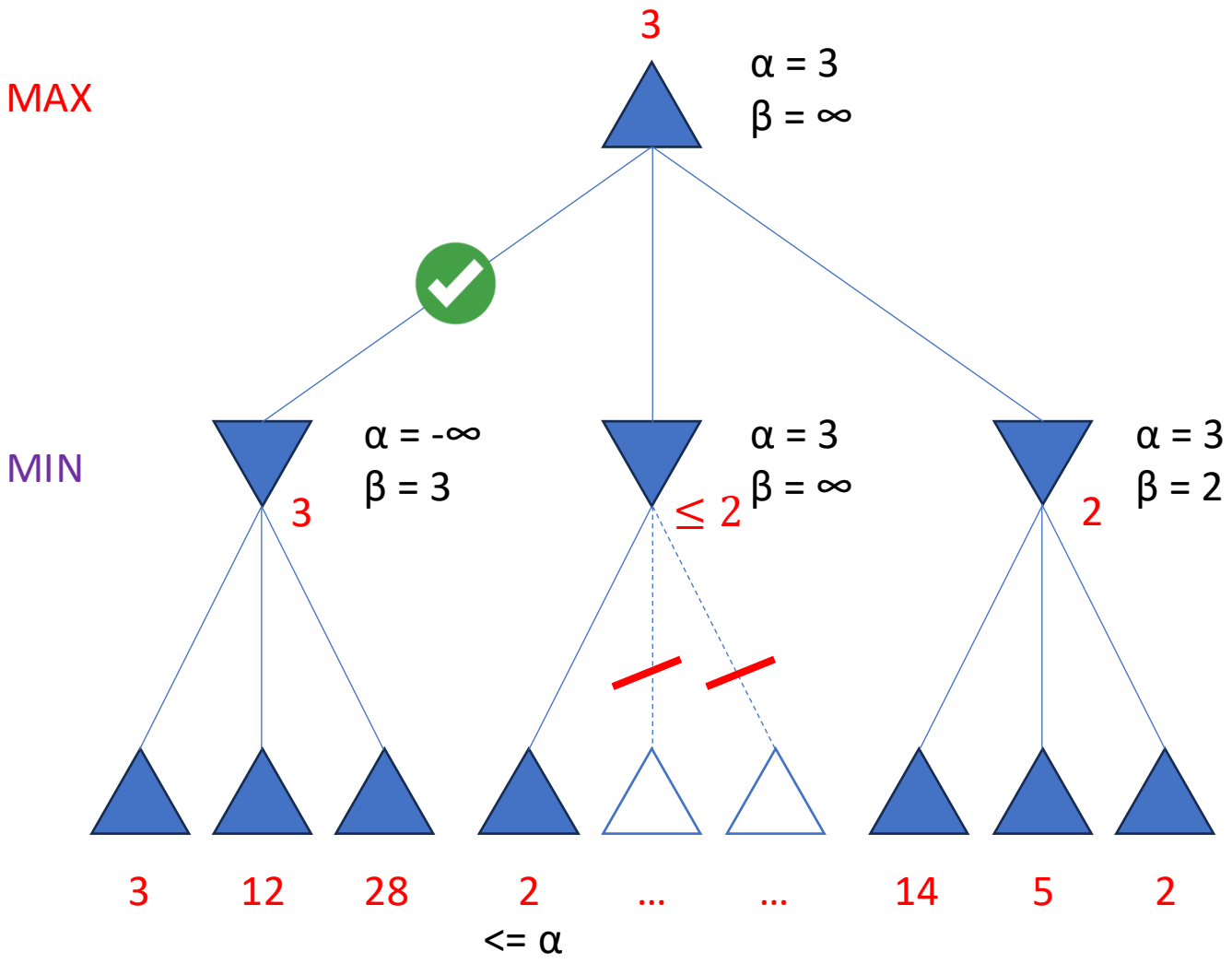
```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```



Alpha-beta Pruning

α = highest value for MAX
β = lowest value for MAX

```
def alpha_beta_search(state):
    (action, v) = max_value(state, -∞, ∞)
    return action

def max_value(state, α, β):
    if is_terminal(state): return utility(state)
    v = -∞
    for next_state in expand(state):
        v = max(v, min_value(next_state, α, β))
        α = max(α, v)
        if v >= β: return v
    return v

def min_value(state , α, β):
    if is_terminal(state): return utility(state)
    v = ∞
    for next_state in expand(state):
        v = min(v, max_value(next_state, α, β))
        β = min(β, v)
        if v <= α: return v
    return v
```



Summary: Local Search

- There are problems with a very large state space
  - Good enough solution is okay
- Local Search
  - Hill-climbing (greedy): **pick the best** among neighbours, repeat
- State space landscape
  - Local maximum
  - Global maximum
  - Shoulder

Summary: Adversarial Search

- Games vs Search Problems:
  - Search problems: **predictable** world, deterministic and stochastic
  - Games: **“unpredictable”** opponent, strategic
- Minimax:
  - Assume **opponent behaves optimally**
  - Pick a move that **maximizes** player’s **utility**
- Alpha-beta pruning
  - **Prune branches** that are useless (**will not change the decision**)
- Handling large/infinite game trees:
  - **Cutoff** the search in the middle of the game and use **evaluation function**